

PTS

June 2, 2026

1 Popis softvérovo intenzívneho systému

1.1 Softvérovo intenzívny systém

Softvérovo intenzívny systém je systém, ktorého podstatná časť funkcionality, správania, hodnoty a zložitosti je realizovaná softvérom. Takýto systém zvyčajne nepozostáva iba zo samotného programu, ale aj z používateľov, externých systémov, hardvéru, dát, procesov, organizačného prostredia a prevádzkových obmedzení.

Pri jeho opise je potrebné zachytiť viacero pohľadov, pretože jeden model nedokáže úplne opísať všetky relevantné vlastnosti systému. Ucelený pohľad na systém vzniká kombináciou viacerých aspektov.

1.2 Aspekty softvérovo intenzívneho systému

Pri opise softvérovo intenzívneho systému je potrebné zachytiť najmä tieto aspekty:

1. Kontext systému

Je potrebné určiť hranice systému, teda čo do systému patrí a čo je už jeho okolie. Zároveň treba identifikovať externých aktérov, používateľov, externé systémy, zariadenia alebo organizácie, ktoré so systémom komunikujú.

Kontext systému odpovedá na otázky:

- Kto alebo čo používa systém?
- Aké externé systémy so systémom komunikujú?
- Kde sú hranice systému?

2. Funkcionalita systému

Funkcionalita opisuje, aké služby má systém poskytovať používateľom alebo iným systémom. Ide o funkčné požiadavky, ktoré určujú, čo má systém robiť.

Príklady funkčných požiadaviek:

- systém umožní zákazníkovi vyhľadávať produkty,
- systém umožní vložiť produkt do nákupného košíka,
- systém umožní vykonať platbu kartou.

3. Správanie systému

Nestačí vedieť iba to, aké funkcie systém poskytuje. Je potrebné opísať aj to, ako systém reaguje na udalosti, v akom poradí prebiehajú činnosti, aké alternatívne scenáre môžu nastať a ako sa systém správa pri chybách.

Tento aspekt zahŕňa:

- scenáre použitia,
- algoritmy a pracovné postupy,
- reakcie na vstupy a udalosti,
- alternatívne a výnimočné toky.

4. Štruktúra systému

Štruktúrny pohľad opisuje, z akých častí sa systém skladá. Môže ísť o triedy, komponenty, balíky, subsystémy, moduly alebo vrstvy architektúry.

Štruktúra systému odpovedá na otázky:

- Aké hlavné časti systém obsahuje?
- Aké sú medzi nimi vzťahy?
- Aké dáta systém spracúva?

5. Dátový aspekt

Softvérový systém často pracuje s významnými dátami. Preto je potrebné zachytiť dôležité dátové entity, ich atribúty, vzťahy a obmedzenia.

Tento aspekt je dôležitý najmä pri informačných systémoch, databázových aplikáciách a podnikových systémoch.

6. Interakcia medzi objektmi alebo komponentmi

Pri zložitejších systémoch je dôležité opísať, ako jednotlivé objekty, komponenty alebo služby spolupracujú pri realizácii konkrétnej funkcionality.

Tento pohľad pomáha pochopiť vnútornú logiku systému.

7. Stavy systému alebo objektov

Niektoré objekty alebo časti systému môžu nadobúdať rôzne stavy. Ich správanie závisí od aktuálneho stavu a udalostí, ktoré nastanú.

Príkladom môže byť objednávka, ktorá môže mať stavy:

- vytvorená,
- zaplatená,

- odoslaná,
- doručená,
- zrušená.

8. Nefunkčné požiadavky

Nefunkčné požiadavky neopisujú priamo funkcionality systému, ale jeho kvalitatívne vlastnosti a obmedzenia.

Patria sem napríklad:

- výkon,
- bezpečnosť,
- spoľahlivosť,
- dostupnosť,
- použiteľnosť,
- škálovateľnosť,
- udržiavateľnosť.

9. Nasadenie systému

Pri reálnych softvérových systémoch je potrebné opísať aj to, na akých fyzických alebo virtuálnych uzloch bude systém nasadený. Ide napríklad o servery, databázové servery, klientské zariadenia, cloudové služby alebo komunikačné siete.

1.3 Použitie UML diagramov

UML, teda Unified Modeling Language, poskytuje súbor diagramov, pomocou ktorých možno zachytiť rôzne pohľady na softvérový systém. UML diagramy sa delia najmä na štruktúrne a behaviorálne diagramy.

1.3.1 Use Case diagram

Use Case diagram patrí medzi behaviorálne UML diagramy. Slúži na zachytenie vysokoúrovňového pohľadu na systém z hľadiska jeho používateľov a externých aktérov.

Zachytáva:

- aktérov,
- prípady použitia,
- hranice systému,
- vzťahy medzi aktérmi a prípadmi použitia,
- vzťahy medzi prípadmi použitia, napríklad `include`, `extend` a generalizáciu.

Aktér predstavuje externú entitu, ktorá so systémom komunikuje. Môže ísť o používateľa, externý systém alebo zariadenie. Prípád použitia predstavuje množinu správania systému, ktorá prináša pozorovateľnú hodnotu pre aktéra alebo zainteresovanú stranu.

Use Case diagram je vhodný najmä na:

- zachytenie rozsahu systému,
- identifikáciu používateľských cieľov,
- komunikáciu so stakeholdermi,
- modelovanie funkčných požiadaviek.

Samotný Use Case diagram však poskytuje iba celkový obraz. Detailnejší opis sa doplní textovým opisom prípadu použitia alebo behaviorálnymi diagramami.

1.3.2 Opis prípadu použitia

Opis prípadu použitia nie je samostatný UML diagram, ale je dôležitou súčasťou use case modelu. Môže mať stručnú alebo podrobnú podobu.

Podrobný, tzv. *fully dressed* opis, typicky obsahuje:

- názov prípadu použitia,
- cieľ,
- rozsah,
- predpodmienky,
- následné podmienky,
- spúšťač,
- primárneho aktéra,
- sekundárnych aktérov,
- hlavný scenár,
- alternatívne a výnimočné scenáre,
- doplňujúce informácie, napríklad prioritu alebo súvisiace nefunkčné požiadavky.

1.3.3 Activity diagram

Activity diagram je behaviorálny UML diagram, ktorý slúži na opis algoritmov, pracovných postupov a tokov činností.

Zachytáva:

- akcie,
- počiatočný a koncový uzol,
- rozhodovanie,
- zlučovanie vetiev,
- paralelizmus pomocou uzlov fork a join,
- objekty a objektové toky,
- swimlane, teda rozdelenie zodpovedností medzi aktérov alebo časti systému.

Activity diagram je vhodný napríklad na:

- detailný opis priebehu prípadu použitia,
- modelovanie podnikových procesov,
- modelovanie algoritmov,
- zobrazenie alternatívnych a paralelných tokov.

1.3.4 State Machine diagram

State Machine diagram, teda stavový diagram, zachytáva životný cyklus objektu alebo časti systému. Zobrazuje stavy, udalosti a prechody medzi stavmi.

Je vhodný najmä pre objekty, ktorých správanie významne závisí od aktuálneho stavu.

Príkladom môže byť objekt *Objednávka*, ktorý môže prechádzať stavmi:

- vytvorená,
- potvrdená,
- zaplatená,
- expedovaná,
- doručená,
- zrušená.

State Machine diagram je vhodný na modelovanie:

- životného cyklu entít,
- reaktívnych systémov,
- stavovo závislého správania,
- protokolov a riadiacich mechanizmov.

1.3.5 Class diagram

Class diagram je štruktúrny UML diagram. Používa sa na opis statickej štruktúry systému.

Zachytáva:

- triedy,
- atribúty,
- operácie,
- asociácie,
- agregácie,
- kompozície,
- dedičnosť,
- rozhrania.

Class diagram je vhodný na:

- modelovanie doménového modelu,
- návrh objektovo orientovanej štruktúry systému,
- zachytenie dátových entít a ich vzťahov,
- podporu návrhu implementácie.

V analytickej fáze sa triedny diagram často používa ako doménový model, ktorý opisuje pojmy z problémovej oblasti. V návrhovej fáze môže byť detailnejší a bližší implementácii.

1.3.6 Sequence diagram

Sequence diagram je behaviorálny UML diagram, ktorý opisuje interakciu objektov alebo komponentov v čase. Zdôrazňuje poradie správ medzi účastníkmi komunikácie.

Zachytáva:

- účastníkov interakcie,
- životné čiary,
- správy,
- poradie volaní,
- návratové správy,
- alternatívy, cykly a podmienky.

Sequence diagram je vhodný na:

- opis realizácie prípadu použitia,
- modelovanie komunikácie medzi objektmi,
- návrh rozhraní medzi komponentmi,
- pochopenie dynamiky systému.

1.3.7 Communication diagram

Communication diagram je podobný sequence diagramu, ale kladie väčší dôraz na väzby medzi objektmi než na časové poradie. Poradie správ sa zvyčajne vyjadruje číslovaním.

Je vhodný vtedy, keď chceme zdôrazniť štruktúru spolupráce medzi objektmi.

1.3.8 Component diagram

Component diagram je štruktúrny UML diagram, ktorý opisuje systém ako súbor komponentov a ich rozhraní.

Zachytáva:

- softvérové komponenty,
- poskytované a požadované rozhrania,
- závislosti medzi komponentmi.

Používa sa najmä pri architektonickom návrhu systému. Pomáha pochopiť, z akých väčších častí sa systém skladá a ako tieto časti spolu komunikujú.

1.3.9 Package diagram

Package diagram slúži na organizáciu modelu alebo systému do balíkov. Zachytáva zoskupenie tried, komponentov alebo iných prvkov a závislosti medzi týmito balíkmi.

Používa sa najmä pri väčších systémoch, kde pomáha zvládnuť zložitosť modelu.

1.3.10 Deployment diagram

Deployment diagram je štruktúrny UML diagram, ktorý opisuje fyzické alebo virtuálne nasadenie systému.

Zachytáva:

- výpočtové uzly,
- servery,

- klientské zariadenia,
- databázové servery,
- artefakty nasadené na uzloch,
- komunikačné spojenia medzi uzlami.

Je vhodný najmä na opis prevádzkovej architektúry systému.

1.4 Prepojenie diagramov

Jednotlivé UML diagramy sa navzájom dopĺňajú. Napríklad Use Case diagram môže zachytiť, že zákazník vykonáva prípad použitia *Checkout*. Detailný priebeh tohto prípadu použitia môže byť opísaný textovo alebo pomocou activity diagramu. Interakcia medzi objektmi pri realizácii objednávky môže byť zachytená sequence diagramom. Triedy ako *Customer*, *Order*, *Product* a *ShoppingCart* môžu byť zachytené class diagramom. Nasadenie aplikácie, databázy a externých služieb môže byť zachytené deployment diagramom.

Preto sa softvérovo intenzívny systém neopisuje jedným diagramom, ale súborom vzájomne konzistentných modelov.

1.5 Zhrnutie

Na vytvorenie uceleného pohľadu na softvérovo intenzívny systém je potrebné zachytiť jeho kontext, funkcionality, správanie, štruktúru, dáta, interakcie, stavy, nefunkčné požiadavky a spôsob nasadenia.

UML poskytuje rôzne diagramy pre rôzne pohľady na systém. Use Case diagram zachytáva funkčné požiadavky a používateľské ciele, activity diagram opisuje pracovné postupy, state machine diagram zachytáva stavovo závislé správanie, class diagram opisuje statickú štruktúru, sequence diagram zobrazuje interakcie v čase, component diagram zachytáva architektúru komponentov a deployment diagram opisuje fyzické nasadenie systému.

Správny opis systému preto vzniká kombináciou viacerých diagramov a textových špecifikácií, pričom každý model zachytáva inú vlastnosť systému.

2 Fázy vývoja softvéru

2.1 Softvérový vývojový proces

Vývoj softvéru sa zvyčajne člení na tieto fázy:

1. **Business analysis,**
2. **Requirements,**
3. **Architecture and design,**
4. **Implementation,**

5. **Verification and validation,**

6. **Maintenance.**

Jednotlivé fázy nemusia vždy prebiehať striktne sekvenčne. V praxi sa môžu prekryvať, opakovať alebo kombinovať podľa zvolenej metodológie vývoja.

2.2 Fázy vývoja softvéru

2.2.1 Business analysis

Business analysis odpovedá na otázku:

Prečo má byť systém vytvorený alebo zmenený?

Cieľom tejto fázy je identifikovať zmeny v organizácii, ktoré sú potrebné na dosiahnutie strategických cieľov. Nemusí ísť iba o IT zmeny, ale aj o zmeny procesov, organizačnej štruktúry, pravidiel alebo stratégie.

Výstupom tejto fázy môže byť napríklad počiatočná projektová dokumentácia, ktorá obsahuje:

- cieľ a účel projektu,
- rozsah projektu,
- kľúčových stakeholderov,
- business case,
- základné obmedzenia a očakávania.

Táto fáza sa často prekryva s analýzou požiadaviek.

2.2.2 Requirements

Fáza požiadaviek odpovedá na otázku:

Čo má nový alebo zmenený systém robiť?

Ide o objavovanie, dokumentovanie a udržiavanie požiadaviek na IT systém. Požiadavky môžu byť funkčné aj nefunkčné.

Funkčné požiadavky opisujú, aké služby a funkcie má systém poskytovať. Napríklad:

- používateľ môže vložiť produkty do nákupného košíka,
- používateľ môže zaplatiť kartou,
- systém zobrazí cenu jednotlivých položiek.

Nefunkčné požiadavky opisujú kvalitatívne vlastnosti systému, napríklad výkon, bezpečnosť, dostupnosť alebo použiteľnosť.

Výstupom tejto fázy je špecifikácia požiadaviek, ktorá môže obsahovať text, modely, diagramy, schémy, user stories alebo use case model.

2.2.3 Architecture and design

Fáza architektúry a návrhu odpovedá na otázku:

Ako má nový alebo zmenený systém fungovať?

Architektúra poskytuje vysokoúrovňové usporiadanie komponentov alebo modulov systému. Návrh poskytuje detailnejšie usporiadanie vo vnútri jednotlivých komponentov alebo modulov.

V tejto fáze sa rozhoduje napríklad o:

- architektonickom štýle,
- rozdelení systému na komponenty,
- použitých technológiách,
- dátovom modeli,
- komunikačných rozhraniach,
- návrhových vzoroch.

Príkladom architektonického rozhodnutia môže byť použitie klient-server architektúry alebo vzoru MVC, teda Model-View-Controller.

Výstupom sú architektonické a návrhové modely a dokumentácia.

2.2.4 Implementation

Implementation je fáza, v ktorej sa návrh premieňa na zdrojový kód. Programátori implementujú požadovanú funkcionality, vytvárajú databázové štruktúry, používateľské rozhrania, integračné rozhrania a ďalšie časti systému.

Táto fáza zahŕňa najmä:

- písanie zdrojového kódu,
- konfiguráciu systému,
- implementáciu databázy,
- integráciu komponentov,
- priebežné testovanie častí systému.

2.2.5 Verification and validation

Verification and validation je fáza overovania a validácie systému.

Verifikácia odpovedá na otázku:

Vytvorili sme systém správne?

Teda či systém zodpovedá špecifikácii, návrhu a technickým požiadavkám.

Validácia odpovedá na otázku:

Vytvorili sme správny systém?

Teda či systém skutočne rieši potreby používateľov a stakeholderov.

Táto fáza zahŕňa napríklad:

- jednotkové testovanie,
- integračné testovanie,
- systémové testovanie,
- akceptačné testovanie,
- kontrolu splnenia požiadaviek.

2.2.6 Maintenance

Maintenance je fáza údržby systému po jeho nasadení. Systém sa monitoruje, opravuje, rozširuje a prispôsobuje novým požiadavkám.

Údržba môže zahŕňať:

- opravu chýb,
- optimalizáciu výkonu,
- aktualizáciu technológií,
- pridávanie novej funkcionality,
- prispôbenie systému zmenám v prostredí.

2.3 Podporné aktivity vývoja softvéru

Popri hlavných fázach vývoja existujú aj podporné aktivity, ktoré prebiehajú naprieč celým projektom.

Medzi najdôležitejšie patria:

- **Project management** – plánovanie, míľniky, termíny, pridelovanie zdrojov, riadenie rizík a sledovanie stavu projektu.
- **Configuration management** – správa a udržiavanie integrity výstupov projektu, napríklad zdrojového kódu, dokumentácie, konfigurácií a verzií.

- **Quality assurance** – zabezpečenie, že požiadavky na kvalitu budú splnené počas celého vývojového procesu.
- **Documentation** – tvorba a údržba technickej, používateľskej a projektovej dokumentácie.

2.4 Najbežnejšie metodológie vývoja softvéru

2.4.1 Waterfall model

Waterfall, teda vodopádový model, je tradičný sekvenčný model vývoja softvéru. Jednotlivé fázy nasledujú za sebou v pevnom poradí:

Business analysis → Requirements → Architecture and design →
Implementation → Verification and validation → Maintenance

Každá fáza sa typicky dokončí pred začiatkom nasledujúcej fázy. Výstupy jednej fázy slúžia ako vstupy pre ďalšiu fázu.

Výhody waterfall modelu:

- jasná štruktúra procesu,
- dobrá plánovateľnosť,
- vhodný pre projekty so stabilnými požiadavkami,
- dôraz na dokumentáciu,
- jednoduché riadenie z pohľadu manažmentu.

Nevýhody waterfall modelu:

- nízka flexibilita pri zmene požiadaviek,
- neskorá spätná väzba od používateľov,
- funkčný softvér vzniká až v neskorších fázach,
- chyby v požiadavkách sa môžu odhaliť príliš neskoro.

Waterfall je vhodný najmä tam, kde sú požiadavky dobre známe, stabilné a projekt je možné dopredu presne naplánovať.

2.4.2 Iteratívno-inkrementálny model

Iteratívno-inkrementálny model rozdeľuje vývoj na viacero iterácií. V každej iterácii sa realizujú viaceré fázy vývoja, napríklad analýza požiadaviek, návrh, implementácia a testovanie.

Výsledkom každej iterácie je inkrement, teda čiastočne funkčná verzia produktu. Finálny produkt vzniká postupným pridávaním ďalších inkrementov.

Výhody iteratívno-inkrementálneho modelu:

- skoršie dodanie časti funkcionality,
- možnosť získať spätnú väzbu od používateľov,
- lepšie zvládanie zmien požiadaviek,
- postupné znižovanie rizika,
- možnosť priebežného testovania.

Nevýhody:

- náročnejšie riadenie projektu,
- potreba dobre plánovať architektúru,
- riziko nekonzistentnosti medzi inkrementmi,
- potreba priebežnej integrácie.

Tento model je vhodný pre projekty, kde sa požiadavky môžu vyvíjať a kde je užitočné dodávať systém po častiach.

2.4.3 Spiral model

Špirálový model kombinuje prvky vodopádového, iteratívneho a rizikovo orientovaného prístupu. Vývoj prebieha v cykloch, pričom každý cyklus obsahuje plánovanie, analýzu rizík, vývoj a vyhodnotenie.

Charakteristickým znakom špirálového modelu je dôraz na riadenie rizík. Na začiatku každého cyklu sa identifikujú a analyzujú riziká a podľa nich sa volí ďalší postup.

Výhody:

- vhodný pre veľké a rizikové projekty,
- dôraz na analýzu rizík,
- umožňuje priebežné spresňovanie požiadaviek,
- podporuje prototypovanie.

Nevýhody:

- proces je zložitejší,
- vyžaduje skúsené riadenie,
- môže byť nákladný,
- nie je vhodný pre malé jednoduché projekty.

2.4.4 Rapid Application Development

Rapid Application Development, skrátene RAD, je metodológia zameraná na rýchly vývoj aplikácií. Využíva prototypovanie, opakovanú spätnú väzbu od používateľov a často aj nástroje na rýchlu tvorbu používateľských rozhraní a aplikácií.

Výhody:

- rýchle vytvorenie prototypu,
- intenzívna spolupráca s používateľmi,
- rýchle overovanie požiadaviek,
- vhodné pre GUI-orientované aplikácie.

Nevýhody:

- menej vhodné pre veľmi veľké alebo kritické systémy,
- riziko slabšej architektúry pri príliš rýchlom vývoji,
- vyžaduje dostupnosť používateľov a stakeholderov.

2.4.5 Agilné metodológie

Agilné metodológie sú založené na iteratívno-inkrementálnom vývoji, spolupráci, flexibilitě, kvalite a neustálom zlepšovaní.

Riešenie sa vyvíja postupne prostredníctvom spolupráce samoorganizujúcich sa a cross-funkčných tímov. Vývoj prebieha v krátkych iteráciách, často nazývaných sprinty.

Medzi známe agilné prístupy patria:

- Scrum,
- Extreme Programming,
- Kanban,
- Lean software development.

Hlavné princípy agilného vývoja:

- flexibilita pri zmenách požiadaviek,
- časté dodávanie funkčného softvéru,
- úzka spolupráca so zákazníkom,
- priebežná spätná väzba,
- dôraz na kvalitu,

- neustále zlepšovanie procesu.

Výhody agilných metodológií:

- rýchla reakcia na zmeny,
- pravidelné dodávanie hodnoty,
- vysoká miera komunikácie,
- lepšia spätná väzba od zákazníka,
- priebežné testovanie a integrácia.

Nevýhody agilných metodológií:

- menej vhodné pri pevne regulovaných projektoch,
- náročné na komunikáciu a disciplínu tímu,
- riziko nedostatočnej dokumentácie,
- problém pri veľmi veľkých tímoch alebo komplexných závislostiach.

2.4.6 Extreme Programming

Extreme Programming, skrátene XP, je agilná metodológia zameraná najmä na technické praktiky vývoja softvéru.

Typické praktiky XP sú:

- test-driven development,
- pair programming,
- continuous integration,
- refactoring,
- jednoduchý dizajn,
- časté releasy,
- kolektívne vlastníctvo kódu.

XP je vhodné najmä tam, kde sa očakávajú časté zmeny požiadaviek a kde je dôležitá vysoká kvalita kódu.

2.4.7 Continuous Integration

Continuous Integration je prístup, pri ktorom sa zmeny v kóde často integrovajú do spoločnej vetvy a automaticky sa zostavujú a testujú.

Cieľom je čo najskôr odhaliť integračné chyby. Continuous Integration sa často používa spolu s agilnými metodológiami a DevOps praktikami.

2.4.8 Lean software development

Lean prístup sa zameriava na elimináciu plytvania, zvyšovanie hodnoty pre zákazníka a neustále zlepšovanie procesu. V softvérovom vývoji znamená napríklad minimalizáciu zbytočnej práce, obmedzenie rozpracovanosti, rýchlu spätnú väzbu a dôraz na kvalitu.

2.5 Použitie UML diagramov v jednotlivých fázach vývoja softvéru

UML diagramy sa môžu používať vo viacerých fázach vývoja softvéru. Ich úloha sa však líši podľa toho, či systém analyzujeme, navrhujeme, implementujeme, testujeme alebo udržiavame.

2.5.1 Business analysis

Vo fáze business analysis sa UML používa skôr obmedzene. Cieľom tejto fázy nie je ešte detailne navrhovať systém, ale pochopiť podnikový problém, procesy a dôvody zmeny.

Vhodné diagramy:

- **Activity diagram** – na zachytenie biznis procesov, pracovných tokov a činností v organizácii.
- **Use Case diagram** – na hrubé vymedzenie očakávaného systému, jeho aktérov a základných cieľov.

Activity diagram môže napríklad zachytiť existujúci proces vybavenia objednávky ešte pred automatizáciou.

2.5.2 Requirements

Vo fáze požiadaviek sa UML používa najmä na zachytenie toho, čo má systém robiť z pohľadu používateľov a externých systémov.

Vhodné diagramy:

- **Use Case diagram** – zachytáva aktérov, prípady použitia, hranice systému a vzťahy medzi nimi.
- **Activity diagram** – opisuje detailný priebeh prípadu použitia alebo workflow.
- **State Machine diagram** – zachytáva stavy významných objektov alebo častí systému.
- **Class diagram na analytickej úrovni** – zachytáva doménový model, teda hlavné pojmy problémovej oblasti a ich vzťahy.

Use Case diagram je vhodný na komunikáciu so stakeholdermi, pretože poskytuje vysokoúrovňový pohľad na funkčnosť systému. Samotný diagram však zvyčajne nestačí, preto sa dopĺňa textovým opisom prípadu použitia, scenármi, alternatívnymi tokmi alebo activity diagramom.

2.5.3 Architecture and design

Vo fáze architektúry a návrhu sa UML používa na opis toho, ako bude systém vnútorne usporiadaný a ako budú jeho časti spolupracovať.

Vhodné diagramy:

- **Class diagram** – detailný návrh tried, atribútov, operácií a vzťahov.
- **Component diagram** – rozdelenie systému na komponenty a opis ich rozhraní a závislostí.
- **Package diagram** – organizácia modelu alebo kódu do balíkov a vrstiev.
- **Sequence diagram** – opis spolupráce objektov alebo komponentov pri realizácii konkrétnej funkcionality.
- **Communication diagram** – alternatíva k sequence diagramu, zdôrazňuje najmä väzby medzi objektmi.
- **State Machine diagram** – detailný návrh stavovo závislého správania.
- **Deployment diagram** – návrh fyzického alebo virtuálneho nasadenia systému.

V tejto fáze sú UML diagramy často technickejšie a slúžia najmä vývojárom, architektom a testerom.

2.5.4 Implementation

Vo fáze implementácie sa UML používa najmä ako podpora programovania. Diagramy môžu slúžiť ako návod pre implementáciu alebo môžu byť generované spätne zo zdrojového kódu.

Vhodné diagramy:

- **Class diagram** – mapovanie návrhu tried na zdrojový kód.
- **Sequence diagram** – pochopenie poradia volaní medzi objektmi alebo službami.
- **Component diagram** – implementácia rozhraní medzi komponentmi.
- **Package diagram** – organizácia kódu do modulov a balíkov.

V agilnom vývoji sa v tejto fáze často používajú len také diagramy, ktoré prinášajú okamžitú hodnotu. UML nemusí byť detailné ani úplné, ale má pomôcť tímu pochopiť návrh.

2.5.5 Verification and validation

Vo fáze verifikácie a validácie UML pomáha overovať, či systém zodpovedá požiadavkám a návrhu.

Vhodné použitie UML:

- **Use Case diagram a opisy prípadov použitia** – slúžia ako základ pre akceptačné testy.
- **Activity diagram** – pomáha odvodiť testovacie scenáre pre hlavné, alternatívne a chybové toky.
- **State Machine diagram** – pomáha testovať prechody medzi stavmi.
- **Sequence diagram** – pomáha testovať interakcie medzi komponentmi alebo službami.
- **Deployment diagram** – pomáha overiť správnosť nasadenia a integračného prostredia.

Napríklad zo stavového diagramu objednávky možno odvodiť testy, ktoré overia, či sa objednávka môže dostať iba do povolených stavov a či neexistujú neplatné prechody.

2.5.6 Maintenance

Vo fáze údržby UML diagramy pomáhajú pochopiť existujúci systém a bezpečne navrhovať jeho zmeny.

Vhodné diagramy:

- **Class diagram** – pochopenie štruktúry existujúceho kódu.
- **Component diagram** – pochopenie závislostí medzi časťami systému.
- **Deployment diagram** – pochopenie prevádzkového prostredia.
- **Sequence diagram** – analýza dopadov zmien na interakcie.
- **Use Case diagram** – identifikácia dopadu zmeny na používateľskú funkcionálnosť.

Pri údržbe je dôležité, aby dokumentácia a modely zostali aktuálne. Neaktuálne UML diagramy môžu byť zavádzajúce.

Fáza vývoja	Vhodné UML diagramy
Business analysis	Activity diagram, jednoduchý Use Case diagram
Requirements	Use Case diagram, Activity diagram, State Machine diagram, analytický Class diagram
Architecture and design	Class diagram, Component diagram, Package diagram, Sequence diagram, Communication diagram, State Machine diagram, Deployment diagram
Implementation	Class diagram, Sequence diagram, Component diagram, Package diagram
Verification and validation	Use Case diagram, Activity diagram, State Machine diagram, Sequence diagram, Deployment diagram
Maintenance	Class diagram, Component diagram, Deployment diagram, Sequence diagram, Use Case diagram

Table 1: Použitie UML diagramov v jednotlivých fázach vývoja softvéru

2.6 Prehľad použitia UML diagramov podľa fáz

2.7 Zhrnutie

Vývoj softvéru sa skladá z viacerých fáz: business analysis, requirements, architecture and design, implementation, verification and validation a maintenance. Business analysis vysvetľuje, prečo má byť systém vytvorený alebo zmenený. Requirements opisujú, čo má systém robiť. Architecture and design opisuje, ako má systém fungovať. Implementation znamená tvorbu kódu. Verification and validation overuje správnosť a vhodnosť systému. Maintenance zabezpečuje prevádzku, opravy a rozvoj systému po nasadení.

Medzi najbežnejšie metodológie vývoja patria waterfall, iteratívno- inkrementálny model, špirálový model, RAD a agilné metodológie ako Scrum alebo Extreme Programming. Waterfall je sekvenčný a vhodný pre stabilné požiadavky. Iteratívno-inkrementálny model dodáva systém po častiach. Agilné metodológie sú založené na flexibilitě, spolupráci, krátkych iteráciách a priebežnej spätnej väzbe.

UML diagramy sa používajú v rôznych fázach vývoja na rôzne účely. V analýze pomáhajú pochopiť procesy a požiadavky, v návrhu opisujú štruktúru a spoluprácu častí systému, pri implementácii pomáhajú vývojárom, pri testovaní slúžia ako základ pre testovacie scenáre a pri údržbe pomáhajú pochopiť existujúci systém a dopady zmien.

3 Požiadavky, analýza požiadaviek, use case diagramy

3.1 Požiadavky

Požiadavka je funkcia, obmedzenie alebo iná vlastnosť, ktorú musí systém poskytovať, aby naplnil potreby stakeholderov. Požiadavky určujú najmä to, **čo má systém robiť**, nie prečo má byť systém vytvorený ani ako má byť implementovaný.

Fáza požiadaviek je súčasťou každého životného cyklu vývoja softvéru. Jej cieľom je premeniť neformálne, neúplné a často nejasné predstavy zákazníka a stakeholderov na dobre definovaný, konzistentný a čo najúplnejší súbor požiadaviek.

Požiadavky slúžia ako vstup pre:

- architektúru a návrh systému,
- implementáciu,
- verifikáciu a validáciu,
- údržbu,
- projektové riadenie,
- dokumentáciu,
- zmluvné vymedzenie rozsahu dodávky.

Výstupom fázy požiadaviek je zvyčajne špecifikácia požiadaviek, označovaná aj ako *Software Requirements Specification* alebo skrátené SRS.

3.2 Analýza požiadaviek

Analýza požiadaviek, nazývaná aj requirements analysis, requirements engineering alebo jednoducho analýza, je systematický proces objavovania, dokumentovania, analyzovania, špecifikovania, validovania a udržiavania požiadaviek na softvérový systém.

Požiadavky definujú:

WHAT – čo má systém robiť.

Neodpovedajú primárne na otázku:

- **WHY** – prečo má byť systém vytvorený; to je úloha business analýzy,
- **HOW** – ako má systém fungovať; to je úloha architektúry a návrhu.

V praxi sa však tieto fázy môžu prekrývať. Napríklad business požiadavky môžu byť výsledkom business analýzy, ale zároveň ovplyvňujú požiadavky na systém. Podobne návrh používateľského rozhrania môže byť súčasťou špecifikácie požiadaviek, hoci čiastočne zasahuje do návrhu.

3.3 Zdroje požiadaviek

Požiadavky nevznikajú iba z rozhovoru so zákazníkom. Majú viacero zdrojov, ktoré treba pri analýze zohľadniť.

Medzi základné zdroje požiadaviek patria:

1. Stakeholderi

Stakeholder je jednotlivец, skupina alebo organizácia, ktorá môže ovplyvniť výsledok projektu alebo môže byť výsledkom projektu ovplyvnená.

Medzi stakeholderov patria napríklad:

- vedenie spoločnosti,
- projektový zákazník,
- koncoví používatelia,
- produktový vlastník alebo produktový manažér,
- projektový manažér,
- analytici,
- UX dizajnéri,
- architekti,
- vývojári,
- testeri,
- dokumentaristi,
- obchodné oddelenie,
- marketing,
- právne oddelenie,
- IT podpora,
- IT prevádzka.

Stakeholderov treba identifikovať čo najskôr. Niektorí stakeholderi sa môžu objaviť až neskôr, čo môže viesť k potrebe prepracovania požiadaviek.

2. Počiatočné projektové dokumenty

Ide napríklad o:

- Project Initiation Document,
- Project Charter,
- Business Case,
- štúdie uskutočniteľnosti,
- zadanie projektu,
- zmluvné alebo obchodné podklady.

3. Informácie o existujúcom systéme

Ak nový systém nahrádza alebo rozširuje existujúci systém, dôležitým zdrojom požiadaviek sú:

- existujúca dokumentácia,
- zdrojový kód,
- databázy,
- používateľské manuály,
- známe problémy a incidenty,
- spätná väzba používateľov.

4. Organizačné štandardy

Požiadavky môžu vyplývať z interných pravidiel organizácie, metodík, bezpečnostných politík, architektonických pravidiel alebo procesných štandardov.

5. Pravidlá a regulácie

Systém musí často spĺňať právne, regulačné alebo odvetvové požiadavky. Príkladmi môžu byť GDPR, účtovné pravidlá, bezpečnostné normy alebo doménovo špecifické predpisy.

6. Doménové informácie

Doména aplikácie prináša vlastné pojmy, pravidlá a obmedzenia. Iné požiadavky bude mať bankový systém, zdravotnícky systém, e-shop alebo systém riadenia vlakov.

Doménové požiadavky sú často veľmi dôležité, pretože odrážajú špecifické vlastnosti prostredia, v ktorom bude systém používaný.

7. Vstupné požiadavky rôznej kvality

Na začiatku projektu môžu existovať neformálne požiadavky, zoznamy želaní, poznámky, e-maily, backlog, prezentácie alebo staršie špecifikácie. Ich kvalita môže byť rôzna, preto ich treba analyzovať, spresniť, doplniť a validovať.

3.4 Metódy zachytávania požiadaviek

Zachytávanie požiadaviek sa označuje aj ako *elicitation*. Ide o získavanie požiadaviek zo stakeholderov a ďalších zdrojov.

Medzi najčastejšie metódy patria:

1. Rozhovory so stakeholdermi

Analytik vedie štruktúrované alebo pološtruktúrované rozhovory s používateľmi, zákazníkmi, manažérmi alebo technickými stakeholdermi. Rozhovory sú vhodné na získanie detailných informácií, potrieb, problémov a očakávaní.

2. Workshopy a spoločné stretnutia

Viacerí stakeholderi sa stretnú spolu s analytikom a diskutujú o cieľoch, procesoch, požiadavkách a prioritách. Výhodou je, že sa môžu rýchlo odhaliť konflikty, nejasnosti a rozdielne predstavy.

3. Pozorovanie používateľov

Analytik sleduje používateľov pri práci. Táto metóda je užitočná najmä vtedy, keď používatelia nedokážu presne opísať, čo robia, alebo keď medzi oficiálnym procesom a reálnou praxou existujú rozdiely.

4. Štúdium dokumentácie

Požiadavky možno získavať aj z existujúcich dokumentov, napríklad zo smerníc, používateľských manuálov, procesných opisov, zmlúv, legislatívy alebo dokumentácie existujúceho systému.

5. Dotazníky

Dotazníky sú vhodné vtedy, keď je potrebné získať informácie od veľkého počtu používateľov. Ich nevýhodou je menšia možnosť doplniť odpovede alebo vysvetliť nejasnosti.

6. Analýza existujúceho systému

Pri náhrade alebo modernizácii systému sa analyzuje existujúci systém, jeho funkcionality, používanie, obmedzenia, chyby a integračné väzby.

7. Prototypovanie a mockupy

Prototypy a mockupy pomáhajú stakeholderom lepšie pochopiť budúci systém. Sú veľmi užitočné najmä pri systémoch s výrazným používateľským rozhraním. Mockup predstavuje vizuálnu reprezentáciu alebo náčrt toho, ako bude výsledný produkt vyzeráť.

8. User stories v agilnom vývoji

V agilných metodikách sa požiadavky často zachytávajú vo forme user stories. Typická šablóna je:

*Ako **rola** chcem **jschopnosť**, aby som **získal prínos**.*

User story je skôr príslubom ďalšej diskusie než úplnou špecifikáciou. Môže byť rozdelená, spresnená a doplnená o akceptačné kritériá.

3.5 Proces analýzy požiadaviek

Proces analýzy požiadaviek možno rozdeliť na niekoľko navzájom prepojených aktivít:

1. Identifikácia stakeholderov

Najprv je potrebné zistiť, kto má byť do analýzy zapojený. Cieľom je identifikovať nielen skupiny stakeholderov, ale aj konkrétnych reprezentantov, s ktorými možno komunikovať.

2. Elicitation – získavanie požiadaviek

Požiadavky sa získavajú pomocou rozhovorov, workshopov, pozorovania, štúdia dokumentácie, analýzy existujúcich systémov a ďalších metód.

Výstupom môžu byť poznámky, zápisy zo stretnutí, náčrty, zoznamy potrieb a neformálne požiadavky.

3. Analysis – analýza požiadaviek

V tejto aktivite sa požiadavky triedia, spresňujú, dopĺňajú a kontrolujú. Analytik hľadá nejasnosti, konflikty, duplicity, chýbajúce informácie a závislosti medzi požiadavkami.

Súčasťou analýzy je:

- dekompozícia požiadaviek,
- odstraňovanie nejednoznačností,
- riešenie konfliktov,
- určovanie priorít,
- posudzovanie realizovateľnosti,
- identifikácia rizík,
- sledovanie väzieb medzi požiadavkami.

4. Specification – špecifikácia požiadaviek

Požiadavky sa zapisujú do vhodnej formy. Môže ísť o štruktúrovaný text, katalóg požiadaviek, UML diagramy, BPMN diagramy, ER diagramy, mockupy alebo user stories.

Výstupom je návrh špecifikácie požiadaviek.

5. Validation – validácia požiadaviek

Validácia overuje, či požiadavky správne zachytávajú potreby stakeholderov. Prebieha napríklad pomocou prezentácie návrhu, revízií, workshopov, rokovaní alebo prezentácie GUI modelov.

Výstupom je finálna špecifikácia požiadaviek.

6. Maintenance – udržiavanie požiadaviek

Požiadavky sa počas projektu menia. Preto je potrebné ich spravovať, verziovať, sledovať ich stav a dopady zmien.

3.6 Typy požiadaviek

Požiadavky možno členiť podľa úrovne abstrakcie a podľa ich charakteru.

3.6.1 Úrovne požiadaviek

Rozlišujeme tri základné úrovne požiadaviek:

1. Business requirements

Business požiadavky opisujú vysokoúrovňové ciele organizácie. Sú písané najmä pre manažment a odpovedajú na otázku, aký obchodný alebo organizačný cieľ má byť dosiahnutý.

Príklady:

- Zvýšiť online predaj o 20 % v nasledujúcom roku.
- Znížiť počet nesprávne spracovaných objednávok o 50 %.
- Zvýšiť počet opakovaných objednávok o 10 %.

Samotné business požiadavky ešte neopisujú riešenie dostatočne.

2. Stakeholder alebo user requirements

Stakeholder požiadavky opisujú potreby používateľov alebo iných stakeholderov. Zvyčajne sú písané v prirodzenom jazyku a môžu byť podporené diagramami.

Príklady:

- Používateľ chce vytvoriť nový používateľský účet.
- Používateľ chce vidieť históriu objednávok.
- Používateľ chce skontrolovať stav objednávky.

Ani táto úroveň ešte nemusí opisovať riešenie dostatočne detailne.

3. Solution alebo system requirements

Systémové požiadavky opisujú funkcie, služby a prevádzkové obmedzenia systému detailne. Sú základom pre návrh systému a môžu byť aj súčasťou zmluvy.

Príklady:

- Systém umožní vytvoriť používateľský účet s atribútmi: e-mail, meno, priezvisko, adresa, telefónne číslo, heslo a časová značka.
- Systém umožní prihlásenie existujúceho používateľa pomocou e-mailovej adresy a hesla.
- Heslo musí mať aspoň 8 znakov a obsahovať aspoň jeden nealfabetický znak.

3.6.2 Typy systémových požiadaviek

Systémové požiadavky delíme najmä na:

1. Funkčné požiadavky

Funkčné požiadavky opisujú služby alebo funkcie, ktoré má systém poskytovať. Určujú, ako má systém reagovať na vstupy a ako sa má správať v konkrétnych situáciách.

Príklady:

- Systém umožní používateľovi vytvoriť objednávku.
- Systém zobrazí históriu objednávok.
- Systém umožní vložiť položku do nákupného košíka.

2. Nefunkčné požiadavky

Nefunkčné požiadavky opisujú obmedzenia a kvalitatívne vlastnosti služieb poskytovaných systémom.

Môžu zahŕňať:

- výkon,
- dostupnosť,
- spoľahlivosť,
- bezpečnosť,
- použiteľnosť,
- lokalizáciu,
- požiadavky na používateľské rozhranie,
- organizačné požiadavky,
- regulačné a legislatívne požiadavky.

Nefunkčné požiadavky možno ďalej členiť napríklad na:

- produktové požiadavky,
- organizačné požiadavky,
- externé požiadavky.

Produktová požiadavka môže určovať napríklad vlastnosti používateľského rozhrania alebo výkon. Organizačná požiadavka môže určovať, že vývojový proces musí dodržiavať internú metodiku firmy. Externá požiadavka môže vyplývať z legislatívy alebo regulácie.

3. Doménové požiadavky

Doménové požiadavky vyplývajú z aplikačnej domény systému. Odrážajú špecifické pravidlá, výpočty, pojmy alebo obmedzenia v danej oblasti.

Príkladom môže byť požiadavka na výpočet brzdné dráhy vlaku podľa fyzikálnych a prevádzkových pravidiel železničnej domény.

3.7 Organizácia požiadaviek v dokumentácii softvérového projektu

Požiadavky môžu byť v dokumentácii organizované rôznymi spôsobmi. Dôležité je, aby bola dokumentácia zrozumiteľná pre cieľové publikum a zároveň použiteľná pre ďalšie fázy vývoja.

3.7.1 Software Requirements Specification

Výstupom fázy požiadaviek je často dokument SRS (*Software Requirements Specification*). Ide o špecifikáciu požiadaviek, ktorá typicky obsahuje štruktúrovaný text podporený modelmi, diagramami, schémami alebo prototypmi.

SRS môže obsahovať napríklad:

- úvod a účel dokumentu,
- rozsah systému,
- definície, skratky a glosár,
- stakeholderov,
- business požiadavky,
- stakeholder požiadavky,
- systémové požiadavky,
- funkčné požiadavky,
- nefunkčné požiadavky,
- doménové požiadavky,
- use case model,
- dátový alebo konceptuálny model,
- mockupy používateľského rozhrania,
- akceptačné kritériá,
- väzby medzi požiadavkami,
- prílohy.

3.7.2 Formy zápisu požiadaviek

Požiadavky možno zapisovať rôznymi spôsobmi:

1. Neštruktúrovaný text

Ide o dlhý naratívny opis. Dnes sa používa skôr zriedkavo, pretože v ňom býva ťažké rozlíšiť konkrétne požiadavky a sledovať ich atribúty, napríklad prioritu, stav alebo odhady.

Môže byť užitočný na vysvetlenie kontextu a vzťahov medzi vyššími úrovňami požiadaviek.

2. Plochý katalóg požiadaviek

Požiadavky sú uvedené ako zoznam alebo tabuľka. Každá požiadavka má identifikátor a môže mať ďalšie atribúty.

Typické atribúty požiadavky:

- ID,
- text požiadavky,
- typ,
- priorita,
- stav,
- odhad,
- zdroj,
- vlastník,
- väzby na iné požiadavky alebo testy.

Výhodou je, že konkrétne požiadavky sa dajú dobre rozlíšiť a sledovať. Nevýhodou je, že vzťahy medzi požiadavkami, najmä medzi business a stakeholder požiadavkami, sa môžu vysvetľovať ťažšie.

3. Kombinácia – štruktúrovaný text

V praxi sa často používa kombinácia textu, tabuliek, diagramov, modelov a obrázkov. Tento prístup umožňuje zachytiť konkrétne požiadavky aj širší kontext a vzťahy medzi nimi.

4. Modelovacie jazyky a diagramy

Špecifikácia požiadaviek môže byť podporená modelovacími jazykmi:

- UML,
- BPMN,
- E-R diagramami,
- GUI modelmi a mockupmi.

UML sa používa najmä na konceptuálny dátový model a use case model.
BPMN sa používa najmä na modelovanie biznis procesov.

5. User stories a produktový backlog

V agilných prístupoch sú požiadavky často organizované v produktovom backlogu ako prioritizovaný zoznam user stories. Scrum predpokladá, že product backlog obsahuje požiadavky na produkt, ale neurčuje presne, odkiaľ tieto požiadavky pochádzajú. Preto býva často potrebná určitá predanalýza mimo samotného Scrumu.

3.7.3 Glosár

Dôležitou súčasťou dokumentácie je glosár. Jeho cieľom je zabezpečiť, aby sa v celej špecifikácii používali rovnaké pojmy pre rovnaké koncepty.

Napríklad treba konzistentne používať jeden z výrazov:

- systém, e-shop, e-commerce store,
- používateľ, zákazník, kupujúci,
- položka, produkt,
- nákupný košík, shopping basket, cart.

Nekonzistentná terminológia môže viesť k nejasnostiam a nesprávnej interpretácii požiadaviek.

3.7.4 Traceability požiadaviek

Traceability znamená sledovateľnosť väzieb medzi požiadavkami a ďalšími artefaktmi projektu.

Typicky sa sledujú väzby:

Business požiadavky → Stakeholder požiadavky → Systémové požiadavky →
Testovacie prípady → Kód

Traceability pomáha odpovedať napríklad na otázky:

- Z akej business požiadavky vyplýva konkrétna systémová požiadavka?
- Ktoré testy overujú danú požiadavku?
- Aký dopad bude mať zmena konkrétnej požiadavky?
- Sú všetky požiadavky pokryté testami?

Dobrá prax je akceptovať iba také požiadavky, ktoré sú sledovateľné.

3.8 Použitie UML pri požiadavkách

UML môže podporovať špecifikáciu požiadaviek najmä dvoma spôsobmi:

1. Konceptuálny dátový model

Konceptuálny model je vysokoúrovňový dátový model, ktorý zachytáva doménové a systémové pojmy a ich vzťahy. V UML sa často modeluje pomocou class diagramu. Zvyčajne je spojený s glosárom.

2. Use case model

Use case model zachytáva funkčné požiadavky. Typicky pozostáva z:

- use case diagramu,
- opisov prípadov použitia,
- prípadne activity diagramov pre detailnejšie scenáre.

3.9 Use case diagramy

Use case diagram je behaviorálny UML diagram, ktorý zachytáva funkčnosť systému z pohľadu jeho používateľov a externých systémov. Poskytuje vysokoúrovňový pohľad na rozsah systému, aktérov, prípady použitia a ich vzťahy.

Use case diagram je vhodný najmä na:

- zachytenie funkčných požiadaviek,
- komunikáciu so stakeholdermi,
- určenie hraníc systému,
- identifikáciu používateľských cieľov,
- vytvorenie prehľadu rozsahu systému.

3.10 Základné elementy use case diagramov

3.10.1 System boundary

System boundary, teda hranica systému, vyjadruje, čo patrí do systému a čo je už mimo neho. V diagrame sa zobrazuje ako obdĺžnik, v ktorom sú umiestnené prípady použitia.

Názov systému sa zvyčajne píše do hornej časti obdĺžnika.

3.10.2 Actor

Aktér predstavuje externú entitu, ktorá interaguje so systémom. Aktéri sú vždy mimo hranice systému.

Aktérom môže byť:

- človek,
- externý systém,
- hardvérové zariadenie,
- organizácia alebo rola.

Aktér nepredstavuje konkrétnu osobu, ale rolu. Jedna fyzická osoba môže vystupovať vo viacerých rolách a jednu rolu môže vykonávať viacero osôb.

Rozlišujeme:

- **Primárny aktér** – má používateľský cieľ, ktorý systém naplňa; typicky inicializuje prípad použitia.
- **Sekundárny aktér** – poskytuje podpornú službu potrebnú na dokončenie prípadu použitia.

Externý systém možno v UML označiť stereotypom <<system>>.

3.10.3 Use case

Use case, teda prípad použitia, predstavuje funkcionality systému. Je to množina správania vykonávaných systémom, ktorá prináša pozorovateľný výsledok hodnotný pre aktéra alebo iného stakeholdera.

V diagrame sa zobrazuje ako elipsa umiestnená vnútri hranice systému.

Názvy prípadov použitia by mali byť zrozumiteľné a konzistentné. Vhodné sú napríklad názvy:

- Create new order,
- View order history,
- Check order status,
- Print order.

3.10.4 Association

Association je vzťah medzi aktérom a prípadom použitia. Vyjadruje, že aktér interaguje so systémom v rámci daného prípadu použitia.

Zobrazuje sa ako plná čiara medzi aktérom a use case.

3.10.5 Generalization

Generalization je vzťah zovšeobecnenia. Môže sa použiť:

- medzi aktérmi,
- medzi prípadmi použitia.

Pri aktéroch znamená, že špecializovaný aktér dedí možnosti všeobecnejšieho aktéra. Pri prípadoch použitia znamená, že špecializovaný use case rozširuje alebo konkretizuje všeobecnejší use case.

3.10.6 Include

Vzťah `<<include>>` znamená, že jeden prípad použitia vždy obsahuje správanie iného prípadu použitia. Zahnutý use case možno chápať ako podprogram alebo spoločnú opakovane používanú časť správania.

Používa sa vtedy, keď viac prípadov použitia zdieľa rovnakú povinnú časť správania.

Príklad:

Create new order <<include>> Log the action

To znamená, že pri vytvorení novej objednávky sa vždy vykoná aj logovanie akcie.

3.10.7 Extend

Vzťah `<<extend>>` znamená, že správanie rozširujúceho prípadu použitia môže byť vložené do správania rozširovaného prípadu použitia. Ide zvyčajne o voliteľné, podmienené alebo výnimočné správanie.

Príklad:

Create new order based on a copy of past order <<extend>> Create new order

To znamená, že vytvorenie objednávky na základe predchádzajúcej objednávky je špeciálne alebo voliteľné rozšírenie základného prípadu vytvorenia novej objednávky.

3.11 Opis prípadu použitia

Use case diagram poskytuje iba prehľad. Detailné správanie sa zachytáva v opise prípadu použitia.

Opis môže obsahovať:

- názov prípadu použitia,
- cieľ,
- predpoklady,

- následné podmienky,
- primárneho aktéra,
- sekundárnych aktérov,
- hlavný scenár,
- alternatívne scenáre,
- výnimočné scenáre,
- súvisiace nefunkčné požiadavky.

Napríklad pre use case *View order history* môže byť cieľom zobrazíť všetky objednávky priradené k danému používateľskému účtu. Predpodmienkou môže byť, že používateľ je prihlásený. Následnou podmienkou je, že systém zobrazí zoznam objednávok alebo informáciu, že žiadne objednávky neexistujú.

3.12 Problémy pri práci s požiadavkami

Pri analýze požiadaviek často vznikajú problémy:

- stakeholderi presne nevedia, čo chcú,
- stakeholderi síce vedia, čo chcú, ale nevedia to opísať,
- stakeholderi opisujú riešenie namiesto skutočnej potreby,
- business požiadavky sú nejasné,
- chýbajú dôležití stakeholderi,
- stakeholderi nie sú dostupní,
- požiadavky sa významne menia neskoro v analýze,
- požiadavky majú slabú sledovateľnosť.

3.13 Odporúčané postupy

Pri práci s požiadavkami je vhodné dodržiavať tieto zásady:

- trvať na jasných business požiadavkách,
- získavať požiadavky od všetkých relevantných stakeholderov,
- brať do úvahy všetky typy požiadaviek,
- vyhýbať sa sivým zónam a nejasnostiam,
- dokumentovať požiadavky presne a konzistentne,
- akceptovať iba sledovateľné požiadavky,
- validovať požiadavky so všetkými stakeholdermi.

3.14 Zhrnutie

Požiadavky opisujú, čo má systém robiť, aké vlastnosti má mať a aké obmedzenia musí spĺňať. Analýza požiadaviek je systematický proces, ktorý zahŕňa identifikáciu stakeholderov, získavanie požiadaviek, ich analýzu, špecifikáciu, validáciu a udržiavanie.

Požiadavky možno deliť na business požiadavky, stakeholder požiadavky a systémové požiadavky. Systémové požiadavky sa ďalej delia na funkčné, nefunkčné a doménové požiadavky.

Požiadavky sa v dokumentácii organizujú napríklad ako SRS, štruktúrovaný text, katalóg požiadaviek, UML modely, mockupy alebo user stories. Dôležitý je glosár a traceability medzi business požiadavkami, stakeholder požiadavkami, systémovými požiadavkami, testami a kódom.

Use case diagramy slúžia na zachytenie funkčných požiadaviek z pohľadu aktérov. Ich základnými elementmi sú aktéri, prípady použitia, hranica systému a vzťahy association, generalization, <<include>> a <<extend>>.

4 Analýza a plánovanie v iteratívno-inkrementálnom vývoji

4.1 Iteratívno-inkrementálny vývoj

Iteratívno-inkrementálny vývoj je spôsob vývoja softvéru, pri ktorom sa systém nevytvára celý naraz, ale postupne po menších častiach. V každej iterácii sa vyberie časť systému, analyzuje sa, navrhne, implementuje, otestuje a prípadne nasadí alebo predvedie zákazníkovi. Výsledkom iterácie je inkrement, teda čiastočne funkčná verzia produktu.

Základná myšlienka je:

vybrať malú časť systému → vytvoriť ju → získať spätnú väzbu → upraviť plán → pokračovať

Na rozdiel od vodopádového modelu sa spätná väzba nezískava až na konci projektu, ale priebežne. To umožňuje skôr odhaliť chyby v požiadavkách, architektúre, implementácii alebo v samotnom pochopení biznis cieľov.

4.2 Iterácia a inkrement

Iterácia je časovo alebo obsahovo ohrozený vývojový cyklus. V iterácii sa vykonávajú viaceré aktivity vývoja, napríklad analýza, návrh, implementácia, testovanie a vyhodnotenie.

Inkrement je výsledok iterácie. Ide o rozšírenie produktu o novú funkcionálnu, technickú schopnosť alebo poznanie.

Nie každý inkrement musí byť okamžite nasaditeľný koncovým používateľom, ale mal by prinášať nejakú hodnotu. Hodnotou môže byť:

- nová používateľská funkcionálna,

- overenie technického rizika,
- overenie architektúry,
- prototyp,
- spätná väzba od zákazníka,
- spresnenie požiadaviek.

4.3 Typický obsah prvých iterácií softvérového projektu

Prvé iterácie majú v iteratívno-inkrementálnom vývoji špeciálny význam. Ich cieľom nie je iba dodať čo najviac funkcionality, ale najmä znížiť neistotu, overiť základné predpoklady a nastaviť smer ďalšieho vývoja.

Typický obsah prvých iterácií zahŕňa:

1. Spresnenie cieľov projektu

Na začiatku projektu nemusí byť úplne jasné, či navrhovaný softvér skutočne dokáže naplniť biznis ciele zákazníka. Preto sa v prvých iteráciách overuje, či zvolený prístup má potenciál priniesť očakávanú hodnotu.

Príklad:

- Cieľom je zvýšiť online predaj.
- Prvá iterácia môže vytvoriť jednoduchý prototyp objednávkového procesu alebo MVP, aby sa zistilo, či zákazníci dokážu systém používať.

2. Identifikácia a spresnenie kľúčových požiadaviek

Požiadavky sú na začiatku často neúplné, nejasné alebo navzájom nekonzistentné. Prvé iterácie pomáhajú odhaliť, čo zákazník skutočne potrebuje.

V tejto fáze sa venuje pozornosť najmä:

- kľúčovým používateľským scenárom,
- najdôležitejším use casom,
- prioritným user stories,
- nejasným požiadavkám,
- požiadavkám s veľkým dopadom na architektúru.

3. Overenie architektúry

Prvé iterácie často obsahujú tzv. architektonický základ systému. Cieľom je overiť, či navrhnutá architektúra dokáže splniť požiadavky.

Overuje sa napríklad:

- či spolu komponenty vedia komunikovať,
- či zvolená technológia vyhovuje,

- či systém zvládne požadované zaťaženie,
- či je možné integrovať externé služby,
- či architektúra podporuje budúce zmeny.

4. Vytvorenie technickej kostry systému

Často sa vytvára tzv. *walking skeleton*, teda minimálna technická kostra systému. Neobsahuje ešte plnú funkcionálnu, ale prepája hlavné vrstvy systému.

Napríklad:

- jednoduché používateľské rozhranie,
- backendová služba,
- databáza,
- základné API,
- automatizovaný build,
- jednoduché testy,
- základné nasadenie.

5. Overenie najväčších rizík

Prvé iterácie by mali riešiť položky, ktoré sú rizikové alebo nejasné. Nemá zmysel najprv implementovať jednoduché a isté časti systému, ak neskôr môže projekt zlyhať na technickom alebo biznis riziku.

Rizikové položky môžu byť:

- technicky neznáma integrácia,
- výkonnostná požiadavka,
- bezpečnostná požiadavka,
- nová technológia,
- nejasný biznis proces,
- neoverená používateľská potreba.

6. Nastavenie vývojového procesu

Na začiatku sa pripravuje aj infraštruktúra a spôsob práce tímu:

- repozitár zdrojového kódu,
- build pipeline,
- automatizované testovanie,
- nástroje na správu backlogu,
- pravidlá komunikácie,
- spôsob plánovania a vyhodnocovania iterácií.

4.4 Ako sa mení typický rozsah iterácie počas vývoja

Rozsah iterácie sa počas projektu mení podľa toho, koľko neistoty v projekte ešte zostáva.

4.4.1 Začiatok projektu

Na začiatku projektu je veľa neznámych. Nie je úplne jasné:

- čo presne má byť vybudované,
- či požiadavky naplnia biznis ciele,
- či je riešenie technicky realizovateľné,
- aké sú najväčšie riziká,
- ako rýchlo tím dokáže dodávať funkcionality.

Preto sú prvé iterácie často zamerané na:

- analýzu a spresnenie požiadaviek,
- prototypovanie,
- MVP,
- overenie architektúry,
- technické experimenty,
- odstránenie najväčších rizík.

Rozsah prvej iterácie môže byť menší alebo experimentálnejší. Jej cieľom nie je maximalizovať počet funkcií, ale maximalizovať užitočnú spätnú väzbu.

4.4.2 Stred projektu

V strede projektu je už zvyčajne jasnejšia architektúra, tím pozná doménu a požiadavky sú lepšie pochopené. Iterácie sa preto viac zameriavajú na pravidelné dodávanie používateľskej funkcionality.

Typický obsah iterácie v strede projektu:

- implementácia prioritných user stories,
- rozširovanie existujúcich use casov,
- dopĺňanie validácií a pravidiel,
- refaktoring,
- integračné testy,
- zber spätnej väzby od zákazníka.

Rozsah iterácie sa stabilizuje. Tím už vie lepšie odhadovať, koľko práce dokáže v iterácii dokončiť.

4.4.3 Koniec projektu

Na konci projektu sa rozsah iterácií často presúva od tvorby novej funkcionality k stabilizácii produktu.

Typický obsah neskorých iterácií:

- oprava chýb,
- optimalizácia výkonu,
- bezpečnostné úpravy,
- dokončenie dokumentácie,
- používateľské testovanie,
- akceptačné testy,
- príprava nasadenia,
- odstránenie technického dlhu.

Na konci projektu môže byť rozsah iterácie menší z pohľadu novej funkcionality, ale väčší z pohľadu testovania, stabilizácie a prípravy produkcie.

4.5 Položky, ktorým treba pri analýze venovať zvýšenú pozornosť

4.5.1 Rizikové položky

Rizikové položky sú také, pri ktorých existuje vysoká neistota alebo možný veľký negatívny dopad.

Spoznáme ich podľa otázok:

- Vieme to vôbec implementovať?
- Vieme to implementovať s danou technológiou?
- Vieme to implementovať dostatočne rýchlo?
- Vieme splniť výkonové, bezpečnostné alebo legislatívne požiadavky?
- Čo sa stane, ak táto položka nebude fungovať?

Príklad:

- integrácia s bankovou bránou,
- požiadavka na 10 000 súčasných používateľov,
- migrácia veľkého množstva dát,
- použitie novej technológie.

4.5.2 Architektonicky významné položky

Niektoré požiadavky majú veľký dopad na architektúru systému. Ak sa zanedbajú, môže byť neskôr veľmi drahé systém prerobiť.

Spoznáme ich tak, že ovplyvňujú:

- rozdelenie systému na komponenty,
- komunikáciu medzi službami,
- databázový model,
- bezpečnostný model,
- škálovanie,
- nasadenie,
- integračné rozhrania.

Príklad:

- požiadavka na offline režim,
- požiadavka na vysokú dostupnosť,
- požiadavka na auditovanie všetkých akcií,
- požiadavka na integráciu s viacerými externými systémami.

4.5.3 Nejasné a neúplné požiadavky

Nejasné požiadavky treba riešiť skoro, pretože môžu viesť k nesprávnemu návrhu alebo implementácii.

Spoznáme ich podľa toho, že:

- rôzni stakeholderi ich vysvetľujú rôzne,
- obsahujú neurčité slová ako rýchly, jednoduchý, intuitívny, bezpečný, efektívny,
- nemajú jasné akceptačné kritériá,
- nevieme ich otestovať,
- nie je jasné, kto ich vlastní.

4.5.4 Položky s vysokou biznis hodnotou

Iteratívny vývoj má prinášať spätnú väzbu a hodnotu čo najskôr. Preto treba venovať pozornosť položkám, ktoré najviac prispievajú k biznis cieľom.

Spoznáme ich podľa otázok:

- Pomáha táto položka naplniť hlavný cieľ projektu?
- Bude ju používať veľa používateľov?
- Prináša priamu hodnotu zákazníkovi?
- Je potrebná pre MVP?

4.5.5 Položky s veľkým počtom závislostí

Niektoré položky blokujú veľkú časť ďalšej práce. Treba ich analyzovať skoro, pretože ovplyvňujú plán ďalších iterácií.

Spoznáme ich podľa toho, že:

- na nich závisí viacero ďalších user stories,
- vyžadujú rozhodnutie o dátovom modeli,
- vyžadujú externé rozhranie,
- ich oneskorenie blokuje tím.

4.5.6 Nefunkčné požiadavky

Nefunkčné požiadavky sa často prejavajú až neskoro, ale môžu zásadne ovplyvniť architektúru. Preto ich treba analyzovať už v skorých iteráciách.

Patria sem najmä:

- výkon,
- bezpečnosť,
- spoľahlivosť,
- dostupnosť,
- škálovateľnosť,
- použiteľnosť,
- udržiavateľnosť.

4.6 Spätná väzba ako základ plánovania

V iteratívno-inkrementálnom vývoji je spätná väzba kľúčová. Rýchla spätná väzba je hodnotná, pretože chybu je jednoduchšie opraviť vtedy, keď sa odhalí skoro. Zároveň pomáha zistiť, či projekt skutočne smeruje k naplneniu svojich cieľov.

Spätná väzba môže odhaliť napríklad:

- že požiadavky nenapĺňajú biznis ciele,
- chybu v požiadavkách,
- neúplné požiadavky,
- chybu v implementácii,
- problém v komunikácii medzi časťami systému,
- že architektúra nedokáže splniť požiadavky.

Nie každá spätná väzba má rovnakú hodnotu. Preto je dôležité rozhodnúť, v akom poradí sa bude aplikácia budovať. Najprv by sa mali riešiť časti, ktoré poskytujú najdôležitejšiu spätnú väzbu.

4.7 Ako sa určuje rozsah iterácie

Rozsah iterácie možno určovať rôznymi prístupmi. Líšia sa podľa toho, či fixujeme čas, rozsah alebo množstvo práce.

4.7.1 Fixný čas, variabilný rozsah

Tento prístup je typický pre agilné metodiky, napríklad Scrum. Iterácia, často nazývaná sprint, má pevne danú dĺžku, napríklad dva týždne. Do iterácie sa vyberie taký rozsah práce, ktorý tím odhaduje, že dokáže v danom čase dokončiť.

Charakteristika:

- čas iterácie je pevný,
- rozsah sa prispôsobuje kapacite tímu,
- položky sa vyberajú z prioritizovaného backlogu,
- po iterácii sa získava spätná väzba,
- plán sa priebežne upravuje.

Výhody:

- pravidelný rytmus práce,
- častá spätná väzba,

- jednoduchšie plánovanie kapacity,
- vhodné pri nejasných alebo meniacich sa požiadavkách.

Nevýhody:

- nie je vopred garantovaný celý rozsah,
- ťažšie sa dopredu určuje konečná cena a termín,
- vyžaduje aktívnu prioritizáciu backlogu.

4.7.2 Fixný rozsah, variabilný čas

V tomto prístupe sa najprv určí, čo má byť v iterácii hotové, a čas sa prispôsobuje potrebám danej funkcionality.

Charakteristika:

- rozsah iterácie je pevný,
- iterácia končí až po dokončení dohodnutého rozsahu,
- čas trvania iterácie môže kolísať.

Výhody:

- jasné, čo bude výsledkom iterácie,
- vhodné pre menšie balíky práce s jasným cieľom,
- použiteľné pri kontraktach alebo míľnikoch orientovaných na rozsah.

Nevýhody:

- nepravidelný rytmus spätnej väzby,
- riziko, že iterácia bude príliš dlhá,
- horšie odhaľovanie problémov včas.

4.7.3 Fixný čas a fixný rozsah

Tento prístup sa snaží vopred pevne určiť čas aj rozsah. Je podobný vodopádovému alebo kontraktovému plánovaniu.

Charakteristika:

- dopredu sa určí, čo bude dodané,
- dopredu sa určí, kedy to bude dodané,
- vyžaduje stabilné a dobre definované požiadavky.

Výhody:

- jednoduchšie zmluvné a manažérske plánovanie,
- jasný rozsah, čas a často aj cena,
- vhodné pri stabilných požiadavkách a nízkej neistote.

Nevýhody:

- ťažko sa reaguje na zmeny,
- pri neznámych požiadavkách je vysoké riziko zlyhania,
- spätná väzba môže prísť príliš neskoro,
- zmeny môžu výrazne zvýšiť cenu alebo znížiť kvalitu.

4.7.4 Fixný čas a fixné náklady, variabilný rozsah

Pri tomto prístupe sa určí dostupný čas a kapacita tímu, teda aj približné náklady. Rozsah sa potom prispôbuje tomu, čo sa v daných obmedzeniach dá dodať.

Tento prístup je vhodný najmä pri adaptívnom plánovaní, keď nie je možné vopred presne určiť celý rozsah.

Výhody:

- kontrola nad časom a rozpočtom,
- možnosť priebežne meniť priority,
- vhodné pre agilný vývoj,
- zákazník môže priebežne rozhodovať, čo je najdôležitejšie.

Nevýhody:

- nemusí byť dodaný celý pôvodne zamýšľaný rozsah,
- vyžaduje dobrú prioritizáciu,
- vyžaduje spoluprácu zákazníka.

4.8 Obsahové určovanie rozsahu iterácie

Z obsahového hľadiska by sa do iterácie mali vyberať položky podľa viacerých kritérií:

1. Biznis hodnota

Prednosť majú položky, ktoré najviac prispievajú k cieľom projektu.

2. Riziko

Rizikové položky treba riešiť skôr, aby sa projekt mohol prípadne upraviť alebo zastaviť včas.

3. Závislosti

Najprv treba riešiť položky, ktoré odblokujú ďalšiu prácu.

4. Spätná väzba

Vyberajú sa položky, ktoré umožnia získať užitočnú spätnú väzbu od zákazníka, používateľov alebo technického tímu.

5. Realizovateľnosť v iterácii

Položky musia byť dostatočne malé a pripravené tak, aby sa dali v rámci iterácie dokončiť.

6. Koherentnosť inkrementu

Položky by spolu mali tvoriť zmysluplný celok. Iterácia by nemala byť iba náhodný zoznam nesúvisiacich úloh.

4.9 Časové a množstevné určovanie rozsahu iterácie

Z časovej a množstevnej stránky možno rozsah iterácie určovať podľa:

- **dĺžky iterácie** – napríklad 1, 2 alebo 4 týždne,
- **kapacity tímu** – koľko ľudí je dostupných,
- **rýchlosti tímu** – koľko práce tím zvyčajne dokončí,
- **odhadov náročnosti** – napríklad story pointy, človekodni,
- **priorít v backlogu**,
- **definition of ready** – či je položka pripravená na implementáciu,
- **definition of done** – čo znamená, že položka je naozaj dokončená.

V agilnom prístupe sa často používa fixná dĺžka iterácie a tím si do nej berie také množstvo položiek, ktoré podľa skúsenosti dokáže dokončiť. Po každej iterácii sa plán upravuje podľa spätnej väzby a skutočnej rýchlosti tímu.

4.10 Prediktívne a adaptívne plánovanie

4.10.1 Prediktívne plánovanie

Prediktívne plánovanie sa snaží vopred odhadnúť čas a peniaze potrebné na dodanie daného rozsahu. Funguje dobre vtedy, keď nie je príliš veľa neznámych a požiadavky sú stabilné.

Typické vlastnosti:

- najprv sa definuje rozsah,
- potom sa odhaduje čas a cena,
- zmeny požiadaviek sú náročnejšie,
- prístup je bližší vodopádovému modelu.

4.10.2 Adaptívne plánovanie

Adaptívne plánovanie sa používa vtedy, keď nie je úplne jasné, čo presne má byť vybudované. Plán sa priebežne prispôsobuje podľa spätnej väzby.

Typické vlastnosti:

- dôraz je na zisťovanie, čo má byť vytvorené,
- rozsah sa mení podľa spätnej väzby,
- vhodné pre iteratívny a agilný vývoj,
- podporuje učenie sa počas projektu.

4.11 Porovnanie prístupov k určeniu rozsahu iterácie

Prístup	Čo je fixné	Čo je variabilné	Vhodné použitie
Fixný čas, variabilný rozsah	Dĺžka iterácie	Množstvo dokončenej funkcionality	Agilný vývoj, Scrum
Fixný rozsah, variabilný čas	Obsah iterácie	Trvanie iterácie	Jasne definované menšie balíky práce
Fixný čas a fixný rozsah	Čas aj rozsah	Kvalita, náklady alebo riziko zmeny	Stabilné požiadavky, nízka neistota
Fixný čas a náklady, variabilný rozsah	Čas a kapacita tímu	Rozsah funkcionality	Adaptívne plánovanie, agilné kontrakty

Table 2: Porovnanie prístupov k určeniu rozsahu iterácie

4.12 Zhrnutie

V iteratívno-inkrementálnom vývoji sa systém buduje postupne po menších častiach. Každá iterácia prináša inkrement a spätnú väzbu. Prvé iterácie sa zameriavajú najmä na spresnenie cieľov, overenie požiadaviek, vytvorenie technickej kostry, overenie architektúry a odstránenie najväčších rizík.

Rozsah iterácie sa počas projektu mení. Na začiatku je viac experimentálny a rizikovo orientovaný, v strede projektu sa stabilizuje a sústreďuje na dodávanie funkcionality, na konci sa presúva k stabilizácii, testovaniu a príprave nasadenia.

Pri analýze treba venovať zvýšenú pozornosť rizikovým, architektonicky významným, nejasným, biznisovo hodnotným a závislosťami blokujúcim položkám. Rozsah iterácie sa určuje podľa priorít, rizík, spätnej väzby, kapacity tímu, odhadov a dĺžky iterácie.

Existujú rôzne prístupy k plánovaniu: prediktívne plánovanie sa snaží odhadnúť čas a cenu pre daný rozsah, zatiaľ čo adaptívne plánovanie priebežne upravuje plán podľa spätnej väzby. Iteratívny a agilný vývoj typicky používajú adaptívne plánovanie, fixnú dĺžku iterácie a variabilný rozsah.

5 Class diagramy, object diagramy a dizajnové princípy

5.1 Class diagram

Class diagram, teda diagram tried, je štruktúrny UML diagram. Zachytáva statickú štruktúru systému, najmä triedy, ich atribúty, operácie a vzťahy medzi triedami.

Trieda je šablóna na vytváranie objektov. Definuje vlastnosti, teda atribúty, a správanie, teda metódy alebo operácie, ktoré budú mať objekty vytvorené podľa tejto triedy.

Class diagram sa používa najmä na:

- modelovanie domény,
- návrh dátovej a objektovej štruktúry systému,
- opis vzťahov medzi triedami,
- podporu architektonického a detailného návrhu,
- komunikáciu medzi analytikmi, architektmi a vývojármi.

5.2 Object diagram

Object diagram, teda diagram objektov, je štruktúrny UML diagram, ktorý zachytáva konkrétnu situáciu v systéme v určitom okamihu. Kým class diagram opisuje všeobecnú štruktúru tried, object diagram ukazuje konkrétne inštancie tried, ich hodnoty a konkrétne väzby medzi objektmi.

Object diagram možno chápať ako snímku systému v čase.

Používa sa najmä na:

- vysvetlenie konkrétneho príkladu štruktúry objektov,
- overenie správnosti class diagramu,
- ilustráciu zložitých väzieb,
- ukážku konkrétneho stavu systému,
- komunikáciu príkladov s vývojármi alebo analytikmi.

5.3 Základné elementy class diagramu

5.3.1 Trieda

Trieda sa v UML zobrazuje ako obdĺžnik rozdelený typicky na tri časti:

1. názov triedy,
2. atribúty,

3. operácie.

Povinný je iba názov triedy. Atribúty a operácie môžu byť vynechané, ak nie sú pre daný pohľad dôležité.

Príklad triedy:

```
Patient
-----
- firstName: String
- lastName: String
- birthDate: Date
-----
+ getAge(): Integer
+ changeAddress(address: Address): void
```

Abstraktná trieda sa v UML označuje kurzívou alebo anotáciou {**abstract**}.

5.3.2 Atribút

Atribút opisuje vlastnosť triedy. V UML sa atribúty zapisujú vo forme:

```
[visibility] [/] name [: type] [[multiplicity]] [= default]
                        [{modifiers}]
```

Príklady:

```
- firstName: String
- middleName: String = ""
- birthDate: Date
- /age: Integer
- diagnosis[1..*]: Diagnosis {unique}
```

Značka / označuje odvodený atribút. To znamená, že hodnota atribútu sa dá vypočítať z iných atribútov. Napríklad vek možno vypočítať z dátumu narodenia.

Viditeľnosť atribútov sa označuje:

- + public,
- - private,
- # protected,
- ~ package.

Multiplicita určuje, koľko hodnôt môže atribút nadobúdať. Napríklad [1] znamená práve jednu hodnotu, [0..1] znamená voliteľnú hodnotu a [1..*] znamená jednu alebo viac hodnôt.

5.3.3 Operácia

Operácia opisuje správanie triedy. Zapisuje sa vo forme:

```
[visibility] name(parameter-list) [: return-type]
```

Príklady:

```
+ toString(): String
+ createWindow(location: Coordinates, container: Container[0..1]): Window
# display()
- hide()
```

Operácia môže mať parametre, návratový typ a rôzne modifikátory. Napríklad modifikátor `{query}` znamená, že operácia nemení stav systému.

5.3.4 Dátové typy

UML pozná vstavané primitívne typy, napríklad:

- Boolean,
- Integer,
- String,
- Real,
- UnlimitedNatural.

Okrem toho možno definovať používateľské dátové typy pomocou stereotypu `<<dataType>>`. Špeciálnym prípadom je primitívny typ `<<primitive>>` a výčtový typ `<<enumeration>>`.

Príklad výčtového typu:

```
<<enumeration>>
BloodGroup
-----
A
B
AB
O
```

Rozdiel medzi dátovým typom a triedou je v identite. Inštancie dátového typu s rovnakou štruktúrou a rovnakými hodnotami sa považujú za rovnaké. Objekty tried môžu mať vlastnú identitu aj vtedy, keď majú rovnaké hodnoty.

5.3.5 Asociácia

Asociácia je vzťah medzi triedami, ktorý znamená, že medzi objektmi týchto tried môžu existovať väzby.

Príklad:

Student ----- Course
 enrolls in

Asociácia môže mať:

- názov,
- konce asociácie,
- roly,
- multiplicity,
- navigovateľnosť,
- vlastníctvo koncov asociácie.

Príklad multiplicití:

Customer 1 ----- 0..* Order

Tento vzťah znamená, že jeden zákazník môže mať nula alebo viac objednávok a každá objednávka patrí jednému zákazníkovi.

V UML sa zvykne používať konvencia, že ak je typ vlastnosti trieda, modeluje sa ako koniec asociácie, zatiaľ čo ak je typ vlastnosti dátový typ, modeluje sa ako atribút. Táto konvencia nie je v UML povinná, ale je praktická.

5.3.6 Self-association

Self-association je asociácia triedy samej so sebou. Používa sa vtedy, keď objekty tej istej triedy môžu byť medzi sebou vo vzťahu.

Príklad:

Person parent 1..2 ----- 0..* child Person

Tento vzťah hovorí, že osoba môže mať rodičov a deti, pričom všetky zúčastnené objekty sú inštancie triedy **Person**.

5.3.7 Agregácia

Agregácia je špeciálny typ asociácie, ktorý vyjadruje slabý vzťah celok–časť. Časť môže existovať nezávisle od celku.

V UML sa označuje prázdny kosoštvorcom na strane celku.

Príklad:

Team <>----- Person

Osoba môže existovať aj bez tímu, preto ide o slabší vzťah.

5.3.8 Kompozícia

Kompozícia je silnejší typ vzťahu celok–časť. Časť typicky nemôže existovať bez celku. Ak sa zruší celok, zvyčajne sa zrušia aj jeho časti.

V UML sa označuje plným kosoštvorcom na strane celku.

Príklad:

Order ----- OrderItem

Položka objednávky nemá zmysel bez objednávky, ku ktorej patrí.

5.3.9 Association class

Association class je trieda reprezentujúca asociáciu. Používa sa vtedy, keď samotný vzťah medzi objektmi má vlastné atribúty alebo operácie.

Príklad:

Student ----- Course
 Enrollment

enrollmentDate
semester

Tu samotné zapísanie študenta na kurz má vlastnosti, napríklad dátum zápisu a semester.

5.3.10 Generalizácia

Generalizácia je taxonomický vzťah medzi všeobecnejšou a špecifickejšou triedou. Špecifická trieda dedí vlastnosti a operácie všeobecnej triedy.

Príklad:

Vehicle
 ^
 |
Car, Bus, Truck

Triedy Car, Bus a Truck sú špecializácie triedy Vehicle. V UML je povolená aj viacnásobná dedičnosť.

5.3.11 Interface

Rozhranie, teda interface, deklaruje koherentnú službu, ktorú môžu triedy implementovať. Interface nie je inštanciovateľný.

V UML sa označuje stereotypom <<interface>>.

Príklad:

```
<<interface>>
Comparable
-----
isEqual(other: Comparable): Boolean
isLess(other: Comparable): Boolean
```

Trieda, ktorá implementuje toto rozhranie, sa zaväzuje poskytnúť dané operácie.

5.3.12 Dependency

Dependency je závislosť medzi prvkami modelu. Znamená, že jeden prvok, nazývaný klient, používa služby alebo funkcionality iného prvku, nazývaného dodávateľ.

Príklady závislosti:

- trieda používa inú triedu ako parameter operácie,
- trieda používa inú triedu ako lokálnu premennú,
- trieda volá statickú službu inej triedy,
- trieda závisí od rozhrania.

5.4 Základné elementy object diagramu

Object diagram zobrazuje konkrétne objekty a konkrétne väzby medzi nimi.

Základné elementy object diagramu sú:

1. Objekt

Objekt je inštancia triedy. Zapisuje sa typicky vo forme:

```
objectName : ClassName
```

Príklad:

```
customer1 : Customer
order42 : Order
```

Názov objektu a názov triedy bývajú podčiarknuté.

2. Slot

Slot predstavuje konkrétnu hodnotu atribútu objektu.

Príklad:

```
customer1 : Customer
-----
firstName = "Jana"
lastName = "Nováková"
```

3. Link

Link je konkrétna väzba medzi objektmi. Je inštanciou asociácie z class diagramu.

Príklad:

```
customer1 : Customer ----- order42 : Order
```

Tento link zobrazuje, že konkrétny zákazník je prepojený s konkrétnou objednávkou.

5.5 Použitie class diagramov na popis softvéru

Class diagramy možno používať na rôznych úrovniach abstrakcie. Materiál rozlišuje tri základné modelovacie perspektívy: konceptuálny alebo doménový model, špecifikačný logický model a implementačný model. Pri modelovaní je dôležité najprv určiť, ktorú perspektívu používame, a následne ju dodržiavať konzistentne.

5.5.1 Konceptuálny alebo doménový model

Konceptuálny model opisuje pojmy z domény. Softvér ako implementácia ešte nemusí byť modelovaný.

Používa sa najmä vo fáze business analýzy a požiadaviek.

Zachytáva:

- doménové pojmy,
- vzťahy medzi pojmami,
- multiplicity,
- prípadne základné atribúty,
- prípadne vysokoúrovňové operácie.

Príklad doménových tried v e-shope:

- Customer,
- Order,
- OrderItem,

- Product,
- ShoppingCart.

Konceptuálny model býva často spojený s glosárom požiadaviek. Pomáha zjednotiť terminológiu a pochopiť problémovú oblasť.

5.5.2 Špecifikačný alebo logický aplikačný model

Špecifikačný model je už zameraný na softvér, ale stále nie je závislý od konkrétnej implementácie.

Zachytáva:

- logické triedy aplikácie,
- abstraktné dátové typy,
- kľúčové operácie,
- hlavné zodpovednosti tried,
- dôležité väzby medzi triedami.

Používa sa najmä pri architektúre a návrhu systému.

5.5.3 Implementačný model

Implementačný model opisuje konkrétnu implementáciu. Je najbližšie zdrojovému kódu.

Zachytáva:

- konkrétne triedy,
- atribúty s implementačnými typmi,
- metódy,
- viditeľnosti,
- rozhrania,
- závislosti,
- implementačné detaily.

Používa sa najmä pri detailnom návrhu a implementácii.

5.6 Použitie object diagramov na popis softvéru

Object diagramy sa používajú ako doplnok ku class diagramom. Kým class diagram ukazuje všeobecný model, object diagram ukazuje konkrétnu inštanciu tohto modelu.

Object diagram je vhodný napríklad vtedy, keď chceme ukázať:

- konkrétnu objednávku s položkami,
- konkrétneho používateľa a jeho košík,
- konkrétnu konfiguráciu objektov,
- príklad väzieb pri komplikovaných multiplicitách,
- testovací scenár alebo stav systému pred vykonaním operácie.

Príklad:

```
customer1 : Customer
-----
name = "Peter"

order1 : Order
-----
status = "created"

item1 : OrderItem
-----
quantity = 2

customer1 ----- order1
order1 ----- item1
```

Takýto diagram pomáha overiť, či class diagram správne umožňuje reprezentovať reálne situácie v systéme.

5.7 Základné princípy objektovo orientovaného dizajnu

Dizajnové princípy pomáhajú vytvárať softvér, ktorý je zrozumiteľný, udržiavateľný, testovateľný a rozširiteľný. Hoci mnohé vznikli v kontexte objektovo orientovaného programovania, dajú sa použiť aj všeobecnejšie.

5.7.1 Abstrakcia

Abstrakcia znamená sústrediť sa na podstatné vlastnosti objektu alebo systému a potlačiť nepodstatné detaily.

V dizajne to znamená, že trieda alebo rozhranie by malo poskytovať jasný model toho, čo objekt robí, bez toho, aby používateľ musel poznať internú implementáciu.

Príklad:

- Používateľ triedy `PaymentService` potrebuje vedieť, že vie vykonať platbu.
- Nemusí vedieť, ako presne komunikuje s platobnou bránou.

5.7.2 Zapuzdrenie

Zapuzdrenie znamená skrytie vnútorného stavu a implementačných detailov objektu. Objekt poskytuje navonok iba kontrolované rozhranie.

V praxi sa často realizuje tak, že atribúty sú súkromné a prístup k nim je možný iba cez metódy.

Výhody zapuzdrenia:

- znižuje závislosti medzi časťami systému,
- chráni invarianty objektu,
- umožňuje meniť implementáciu bez zmeny verejného rozhrania,
- zlepšuje udržiavateľnosť.

5.7.3 Modularita

Modularita znamená rozdelenie systému na menšie, zmysluplné a relatívne samostatné časti.

Dobrý modul:

- má jasnú zodpovednosť,
- má dobre definované rozhranie,
- skrýva interné detaily,
- možno ho meniť s minimálnym dopadom na zvyšok systému.

V objektovo orientovanom dizajne môžu byť modulmi triedy, balíky, komponenty alebo služby.

5.7.4 Nízka väzba

Väzba, teda coupling, vyjadruje mieru závislosti medzi časťami systému. Cieľom je nízka väzba.

Nízka väzba znamená, že zmena jednej triedy má malý dopad na ostatné triedy.

Príklady, ako znížiť väzbu:

- programovať voči rozhraniam, nie konkrétnym implementáciám,
- nepoužívať zbytočné globálne dáta,
- obmedziť počet priamych závislostí,

- používať dependency injection,
- nevystavovať vnútorné dáta objektu.

5.7.5 Vysoká súdržnosť

Súdržnosť, teda cohesion, vyjadruje, ako úzko spolu súvisia zodpovednosti jednej triedy alebo modulu. Cieľom je vysoká súdržnosť.

Trieda s vysokou súdržnosťou má jednu jasnú zodpovednosť a jej metódy spolu logicky súvisia.

Príklad zlej súdržnosti:

```
UserManager
- createUser()
- sendEmail()
- generateInvoice()
- exportReport()
```

Táto trieda robí príliš veľa nesúvisiacich vecí.

Lepší návrh:

```
UserService
EmailService
InvoiceService
ReportExporter
```

5.7.6 Separation of Concerns

Separation of Concerns znamená oddelenie rôznych zodpovedností systému. Každá časť systému by mala riešiť jednu oblasť problému.

Príklad:

- používateľské rozhranie rieši prezentáciu,
- aplikačná logika rieši use cases,
- doménová logika rieši pravidlá domény,
- dátová vrstva rieši perzistenciu.

Tento princíp je základom viacvrstvových architektúr aj vzoru MVC.

5.7.7 DRY

DRY znamená *Don't Repeat Yourself*. Rovnaká znalosť alebo pravidlo by nemalo byť v systéme zapísané na viacerých miestach.

Duplicita spôsobuje problém pri zmene. Ak sa pravidlo zmení, treba ho upraviť na viacerých miestach a ľahko vznikne nekonzistentnosť.

DRY neznamená, že každý podobný kód sa musí okamžite zovšeobecniť. Dôležité je najmä nezdvojiť rovnakú doménovú alebo biznis znalosť.

5.7.8 KISS

KISS znamená *Keep It Simple*. Návrh by mal byť čo najjednoduchší, ale stále dostatočný pre riešený problém.

Zbytočne komplikovaný návrh:

- je ťažšie pochopiteľný,
- je ťažšie testovateľný,
- zvyšuje náklady na údržbu,
- môže zavádzať predčasnú abstrakciu.

5.7.9 YAGNI

YAGNI znamená *You Aren't Gonna Need It*. Nemali by sme implementovať funkcionality alebo flexibilitu, ktorú aktuálne nepotrebujeme iba preto, že by sa mohla hodiť niekedy v budúcnosti.

Cieľom je vyhnúť sa predčasnej komplexite.

5.8 SOLID princípy

SOLID je skupina piatich často používaných princípov objektovo orientovaného dizajnu.

5.8.1 Single Responsibility Principle

Single Responsibility Principle hovorí, že trieda by mala mať jednu hlavnú zodpovednosť, respektíve jeden dôvod na zmenu.

Ak trieda rieši viac nezávislých zodpovedností, zmeny v jednej oblasti môžu ovplyvniť inú oblasť.

Príklad:

```
OrderService
- calculateTotal()
- saveToDatabase()
- sendConfirmationEmail()
```

Lepšie rozdelenie:

```
OrderCalculator
OrderRepository
EmailService
```

5.8.2 Open/Closed Principle

Open/Closed Principle hovorí, že softvérové entity by mali byť otvorené pre rozšírenie, ale uzavreté pre modifikáciu.

To znamená, že nové správanie by malo byť možné pridať bez úpravy existujúceho overeného kódu, napríklad pomocou polymorfizmu, rozhraní alebo kompozície.

5.8.3 Liskov Substitution Principle

Liskov Substitution Principle hovorí, že objekt odvodeného typu musí byť použiteľný všade tam, kde sa očakáva objekt bazového typu, bez porušenia správnosti programu.

Ak podtrieda mení význam operácií tak, že klientský kód prestane fungovať, dedičnosť je navrhnutá zle.

Príklad problému:

- Ak trieda **Square** dedí od **Rectangle**, ale zmení správanie nastavovania šírky a výšky, môže porušiť očakávania klienta.

5.8.4 Interface Segregation Principle

Interface Segregation Principle hovorí, že klienti by nemali byť nútení závisieť od metód, ktoré nepoužívajú.

Namiesto jedného veľkého rozhrania je lepšie mať viac menších a špecifických rozhraní.

Zlé rozhranie:

```
Machine
- print()
- scan()
- fax()
```

Lepšie:

```
Printer
Scanner
Fax
```

5.8.5 Dependency Inversion Principle

Dependency Inversion Principle hovorí, že vysokoúrovňové moduly by nemali závisieť od nízkoúrovňových modulov. Obe úrovne by mali závisieť od abstrakcií.

Zároveň abstrakcie nemajú závisieť od detailov; detaily majú závisieť od abstrakcií.

Príklad:

```
OrderService --> PaymentGateway interface
CardPaymentGateway implements PaymentGateway
PayPalPaymentGateway implements PaymentGateway
```

Trieda `OrderService` nezávisí od konkrétnej platobnej brány, ale od rozhrania.

5.9 Dedičnosť verzus kompozícia

Dedičnosť umožňuje zdieľať vlastnosti a správanie cez vzťah *is-a*. Napríklad `Car` je `Vehicle`.

Kompozícia znamená skladanie objektov cez vzťah *has-a*. Napríklad `Order` má položky `OrderItem`.

V dizajne sa často odporúča preferovať kompozíciu pred dedičnosťou, ak vzťah *is-a* nie je jasný. Kompozícia býva flexibilnejšia a menej viaže triedy navzájom.

5.10 Zhrnutie

Class diagram je štruktúrny UML diagram, ktorý opisuje statickú štruktúru systému pomocou tried, atribútov, operácií a vzťahov. Základnými elementmi sú trieda, atribút, operácia, dátový typ, asociácia, agregácia, kompozícia, association class, generalizácia, interface a dependency.

Object diagram zobrazuje konkrétne objekty, ich hodnoty a konkrétne väzby v určitom okamihu. Je užitočný najmä na ilustráciu príkladov a overenie správnosti class diagramu.

Class diagramy možno používať ako konceptuálny doménový model, logický aplikačný model alebo implementačný model. Pri modelovaní je dôležité zvoliť správnu perspektívu a dodržiavať ju konzistentne.

Medzi základné dizajnové princípy patria abstrakcia, zapuzdrenie, modularita, nízka väzba, vysoká súdržnosť, separation of concerns, DRY, KISS, YAGNI a princípy SOLID. Ich cieľom je vytvárať softvér, ktorý je zrozumiteľný, udržiavateľný, rozšíriteľný a testovateľný.

6 Základné návrhové vzory

6.1 Návrhový vzor

- poskytujú znovupoužiteľné riešenia návrhových problémov,
- zavádzajú spoločnú terminológiu pre komunikáciu o návrhu,
- uľahčujú pochopenie riešenia, pretože známy vzor je ľahšie rozpoznateľný,
- poskytujú inšpiráciu aj vtedy, keď konkrétny problém presne nezodpovedá známemu vzoru,
- pomáhajú vytvárať flexibilnejší a udržiavateľnejší návrh.

Klasické návrhové vzory podľa skupiny Gang of Four sa delia na tri hlavné skupiny:

1. **vytvárajúce vzory** – *creational patterns*,
2. **štrukturálne vzory** – *structural patterns*,
3. **vzory správania** – *behavioral patterns*.

Okrem nich existujú aj ďalšie skupiny, napríklad vzory pre súbežnosť, doménovo špecifické vzory alebo architektonické vzory.

6.2 Vytvárajúce návrhové vzory

Vytvárajúce návrhové vzory riešia problém vytvárania objektov. Ich cieľom je oddeliť klientsky kód od konkrétneho spôsobu vytvárania objektov.

Pomáhajú v situáciách, keď:

- klient nemá poznať konkrétnu implementačnú triedu,
- spôsob vytvárania objektu je zložitý,
- vytváranie objektu má byť konfigurovateľné,
- chceme kontrolovať počet vytvorených inštancií,
- vytváranie objektov je drahé.

6.2.1 Factory Method

Factory Method je metóda triedy, ktorá vracia nové inštancie inej triedy. Môže byť statická, ale nemusí. Nie je to konštruktor.

Táto metóda môže podľa vstupných parametrov alebo konfigurácie vracaať inštancie rôznych tried. Klient nemusí poznať presnú triedu vytvoreného objektu, stačí mu poznať rozhranie, ktoré objekt poskytuje.

Použitie:

- keď nechceme viazať klienta na konkrétnu triedu,
- keď sa má typ vytváraného objektu meniť podľa konfigurácie,
- keď chceme nahradiť vytváranie objektov injektovanou factory.

Príklad: Graf môže byť reprezentovaný maticou susednosti alebo zoznamom susedov. Trieda, ktorá pracuje s grafom, nemusí vedieť, ktorú reprezentáciu použiť. Použije rozhranie **GraphFactory**, ktoré vytvorí objekt implementujúci rozhranie **Graph**.

6.2.2 Abstract Factory

Abstract Factory je rozhranie obsahujúce viacero súvisiacich factory metód. Používa sa, keď potrebujeme vytvárať rodinu súvisiacich objektov bez toho, aby klient poznal ich konkrétne triedy.

Použitie:

- keď existujú viaceré varianty rodiny objektov,
- keď objekty musia byť navzájom kompatibilné,
- keď chceme jednoducho vymeniť celú rodinu implementácií.

Príkladom môže byť GUI knižnica, kde jedna factory vytvára tlačidlá, textové polia a okná pre Windows a iná factory vytvára tie isté prvky pre Linux alebo macOS.

6.2.3 Builder

Builder je trieda, ktorá sa používa na postupné vytváranie alebo modifikáciu inštancie inej triedy. Používa sa najmä vtedy, keď je konštrukcia objektu zložitá.

Ak by jedna trieda obsahovala metódy pre fázu budovania objektu aj metódy pre jeho bežné používanie, mohla by porušovať princíp jednej zodpovednosti. Builder tieto zodpovednosti oddeľuje.

Použitie:

- keď objekt vzniká vo viacerých krokoch,
- keď existuje veľa voliteľných parametrov,
- keď je dočasná štruktúra počas budovania odlišná od finálneho objektu,
- keď chceme oddeliť konštrukciu od používania objektu.

Príkladom je `StringBuilder` v Jave.

6.2.4 Object Pool

Object Pool rieši situáciu, keď je vytváranie a ničenie objektov drahé. Namiesto toho, aby sa objekty stále vytvárali a rušili, berú sa z poolu a po použití sa do neho vracajú.

Použitie:

- databázové spojenia,
- vlákna,
- sieťové spojenia,
- objekty, ktorých vytvorenie je nákladné.

Typické príklady sú `ThreadPool` a `ConnectionPool`.

6.2.5 Prototype

Prototype vytvára nové objekty kopírovaním existujúceho prototypu. Namiesto vytvárania objektu od začiatku sa skopíruje už pripravená inštancia.

Použitie:

- keď je vytvorenie objektu drahé,
- keď chceme vytvárať podobné objekty,
- keď je jednoduchšie objekt naklonovať než vytvoriť cez konštruktor.

6.2.6 Singleton

Singleton zabezpečuje, že trieda má iba jednu inštanciu, a poskytuje k nej globálny prístupový bod.

Použitie:

- centrálna konfigurácia,
- logovanie,
- správa zdrojov,
- objekty, ktoré majú existovať iba raz.

Singleton treba používať opatrne, pretože môže zavádzať globálny stav a zhoršovať testovateľnosť systému.

6.3 Štrukturálne návrhové vzory

Štrukturálne vzory riešia skladanie tried a objektov do väčších štruktúr. Ich cieľom je dosiahnuť flexibilnú kompozíciu objektov a zjednodušiť vzťahy medzi časťami systému.

6.3.1 Decorator

Decorator umožňuje dynamicky pridať objektu nové správanie bez zmeny jeho triedy. Objekt je obalený dekorátorom, ktorý má rovnaké rozhranie ako pôvodný objekt a deleguje naň volania.

Použitie:

- keď chceme pridávať zodpovednosti objektu dynamicky,
- keď nechceme vytvárať veľa podtried,
- keď potrebujeme kombinovať rôzne rozšírenia správania.

Príkladom môže byť pridanie kompresie, šifrovania alebo bufferovania nad dátovým tokom.

6.3.2 Composite

Composite umožňuje skladať objekty do stromovej štruktúry a pracovať s jednotlivými objektmi aj skupinami objektov jednotným spôsobom.

Použitie:

- stromové štruktúry,
- súborové systémy,
- grafické používateľské rozhrania,
- organizačné štruktúry.

Príklad: adresár obsahuje súbory aj ďalšie adresáre. Klient môže jednotne pracovať so súborom aj adresárom ako s položkou súborového systému.

6.3.3 Facade

Facade poskytuje jednoduché rozhranie k zložitému subsystému. Klient nemusí poznať množstvo tried a detailov subsystému, ale komunikuje s jednou fasádou.

Použitie:

- zjednodušenie práce so zložitým subsystémom,
- oddelenie klienta od vnútorných detailov,
- vytvorenie jednotného vstupného bodu do subsystému.

Príkladom môže byť trieda `OrderFacade`, ktorá zjednodušuje prácu s objednávkami, platbami, skladoom a notifikáciami.

6.3.4 Adapter

Adapter prispôsobuje rozhranie existujúcej triedy tak, aby vyhovovalo rozhraniu očakávanému klientom. Používa sa pri integrácii nekompatibilných rozhraní.

Použitie:

- keď chceme použiť existujúcu triedu, ale jej rozhranie nevyhovuje,
- pri integrácii externých knižníc,
- pri migrácii starého systému na nové rozhranie.

6.3.5 Proxy

Proxy je objekt, ktorý zastupuje iný objekt. Má rovnaké alebo podobné rozhranie ako skutočný objekt, ale pridáva dodatočnú logiku.

Typy proxy:

- **Access proxy** – kontroluje prístup,
- **Remote proxy** – zastupuje objekt v inom procese alebo na inom stroji,
- **Virtual proxy** – odkladá vytvorenie drahého objektu, kým nie je skutočne potrebný.

6.3.6 Flyweight

Flyweight znižuje pamäťovú náročnosť tým, že oddeľuje spoločný stav zdieľaný mnohými objektmi od stavu špecifického pre konkrétnu inštanciu.

Použitie:

- keď existuje veľa podobných objektov,
- keď veľká časť ich stavu je rovnaká,
- keď chceme znížiť spotrebu pamäte.

Príkladom môže byť reprezentácia znakov v textovom editore alebo dlaždíc v hre.

6.4 Vzory správania

Vzory správania opisujú komunikáciu medzi objektmi a rozdelenie zodpovedností pri vykonávaní algoritmov alebo reakcií na udalosti.

6.4.1 Iterator

Iterator poskytuje spôsob, ako postupne prechádzať prvky kolekcie bez toho, aby klient poznal jej vnútornú reprezentáciu.

Použitie:

- prechod zoznamom, stromom alebo inou kolekciou,
- oddelenie algoritmu prechodu od dátovej štruktúry,
- poskytnutie jednotného rozhrania pre rôzne kolekcie.

6.4.2 Observer

Observer umožňuje objektom zaregistrovať sa na odber zmien iného objektu. Keď sa sledovaný objekt zmení, automaticky notifikuje pozorovateľov.

Použitie:

- event systémy,
- používateľské rozhrania,
- publish-subscribe mechanizmy,
- notifikácie pri zmene stavu objektu.

6.4.3 Strategy

Strategy zapuzdruje algoritmus do samostatného objektu. Klient môže používať rôzne stratégie bez toho, aby menil svoj vlastný kód.

Použitie:

- keď existuje viacero variantov algoritmu,
- keď chceme algoritmus meniť za behu,
- keď chceme odstrániť rozsiahle podmienky alebo **switch** príkazy.

Príkladom môže byť výber spôsobu výpočtu ceny, stratégie radenia alebo spôsobu platby.

6.4.4 Template Method

Template Method definuje kostru algoritmu v abstraktnej triede. Niektoré kroky algoritmu sú implementované v abstraktnej triede a iné sú ponechané na podtriedy.

Použitie:

- keď viaceré algoritmy majú rovnakú štruktúru,
- keď sa líšia iba niektoré kroky,
- keď chceme zabrániť duplikácii spoločnej časti algoritmu.

6.4.5 Null Object

Null Object poskytuje rozumný predvolený objekt namiesto hodnoty **null**. Tento objekt implementuje rovnaké rozhranie ako reálny objekt, ale jeho metódy typicky nerobia nič alebo vracajú bezpečné predvolené hodnoty.

Použitie:

- keď chceme odstrániť opakované kontroly na **null**,
- keď existuje prirodzené prázdne správanie,
- keď chceme zjednodušiť klientsky kód.

6.4.6 Memento

Memento umožňuje uložiť a neskôr obnoviť stav objektu bez porušenia jeho zapuzdrenia.

Použitie:

- undo/redo mechanizmy,
- ukladanie histórie stavov,
- návrat objektu do predchádzajúceho stavu.

6.4.7 Visitor

Visitor reprezentuje operáciu vykonávanú nad prvkami objektovej štruktúry. Umožňuje pridať novú operáciu bez zmeny tried prvkov, nad ktorými sa operácia vykonáva.

Použitie:

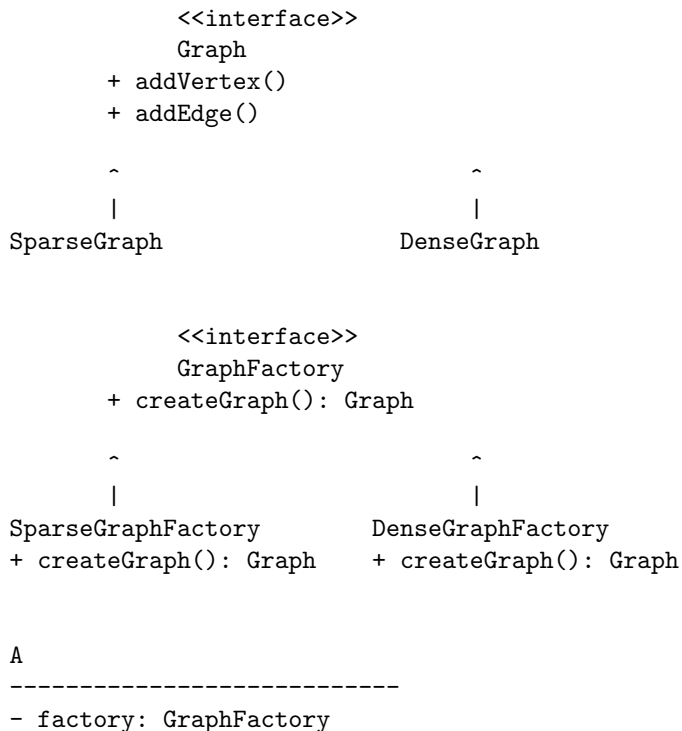
- keď máme stabilnú štruktúru tried, ale často pribúdajú nové operácie,
- pri prechode stromovou štruktúrou,
- pri interpretovaní alebo generovaní kódu,
- pri spracovaní abstraktného syntaktického stromu.

6.5 UML opis vybraných návrhových vzorov

V nasledujúcich častiach sú pomocou textovej UML notácie opísané vybrané návrhové vzory. Na skúške je vhodné vedieť nakresliť aspoň niekoľko z nich: Factory Method, Builder, Decorator, Adapter, Composite, Observer a Strategy.

6.6 Factory Method – UML opis

Factory Method oddeľuje klienta od konkrétnej triedy vytváraného objektu.



```
-----  
+ A(factory: GraphFactory)  
+ doSomething()
```

Vysvetlenie:

- Graph je spoločné rozhranie pre graf.
- SparseGraph a DenseGraph sú konkrétne implementácie.
- GraphFactory definuje factory metódu `createGraph()`.
- SparseGraphFactory vytvára riedky graf.
- DenseGraphFactory vytvára hustý graf.
- Trieda A používa iba rozhranie `GraphFactory`, preto nemusí poznať konkrétnu triedu vytvoreného grafu.

Výhodou je, že trieda A je nezávislá od konkrétnej implementácie grafu. Ak chceme zmeniť typ vytváraného grafu, injektujeme inú factory.

6.7 Builder – UML opis

Builder oddeľuje zložitý proces konštrukcie objektu od samotného výsledného objektu.

Graph

```
-----  
+ addVertex()  
+ addEdge()  
+ getVertices()  
+ getEdges()
```

GraphBuilder

```
-----  
- temporaryVertices  
- temporaryEdges  
-----  
+ addVertex()  
+ addEdge()  
+ replaceEdgeByPath()  
+ build(): Graph
```

Vysvetlenie:

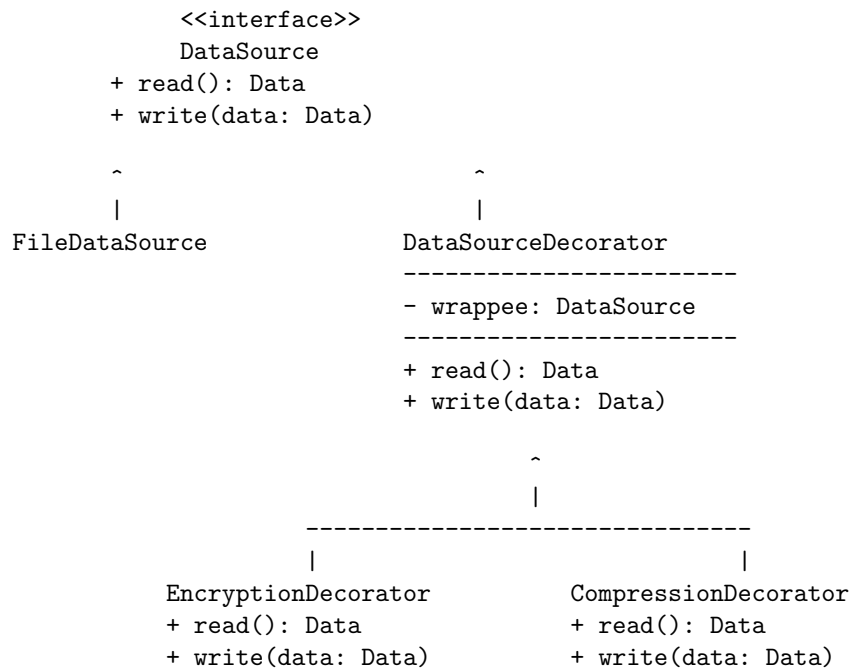
- Graph je výsledný objekt používaný klientom.
- GraphBuilder slúži na postupné vytvorenie alebo úpravu grafu.

- Počas budovania môže mať builder inú vnútornú štruktúru než finálny graf.
- Metóda `build()` vytvorí korektný objekt typu `Graph`.

Builder je vhodný vtedy, keď by priama modifikácia finálneho objektu bola zložitá alebo by porušovala jeho invarianty.

6.8 Decorator – UML opis

Decorator umožňuje pridávať objektu nové správanie obalením pôvodného objektu.



Vysvetlenie:

- `DataSource` je spoločné rozhranie.
- `FileDataSource` je pôvodný objekt.
- `DataSourceDecorator` obsahuje referenciu na iný objekt typu `DataSource`.
- Konkrétne dekorátory, napríklad `EncryptionDecorator` a `CompressionDecorator`, pridávajú nové správanie pred alebo po delegovaní volania na obalený objekt.

Decorator je alternatívou k vytváraniu veľkého množstva podtried pre všetky kombinácie správania.

6.9 Adapter – UML opis

Adapter prispôsobuje rozhranie existujúcej triedy rozhraniu, ktoré očakáva klient.

```
Client
-----
- target: Target
-----
+ execute()
```

```
        <<interface>>
        Target
+ request()
```

```
Adaptee
-----
+ specificRequest()
```

```
Adapter
-----
- adaptee: Adaptee
-----
+ request()
```

Vysvetlenie:

- Klient očakáva rozhranie `Target`.
- Existujúca trieda `Adaptee` má nekompatibilnú metódu `specificRequest()`.
- `Adapter` implementuje `Target` a vo svojej metóde `request()` volá `specificRequest()` na objekte `Adaptee`.

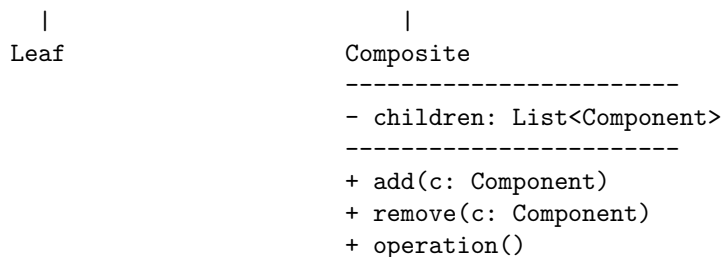
Adapter umožňuje znovu použiť existujúcu triedu bez jej úpravy.

6.10 Composite – UML opis

Composite umožňuje pracovať s jednotlivými objektmi a ich skupinami jednotne.

```
        <<interface>>
        Component
+ operation()
```

```
~ ~
```



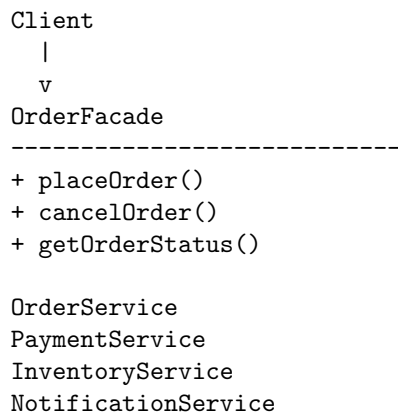
Vysvetlenie:

- **Component** definuje spoločné rozhranie.
- **Leaf** predstavuje jednoduchý objekt bez potomkov.
- **Composite** obsahuje kolekciu objektov typu **Component**.
- Klient môže volať **operation()** rovnako na liste aj na celku.

Tento vzor sa hodí najmä pre stromové štruktúry.

6.11 Facade – UML opis

Facade poskytuje jednoduché rozhranie k zložitému subsystému.



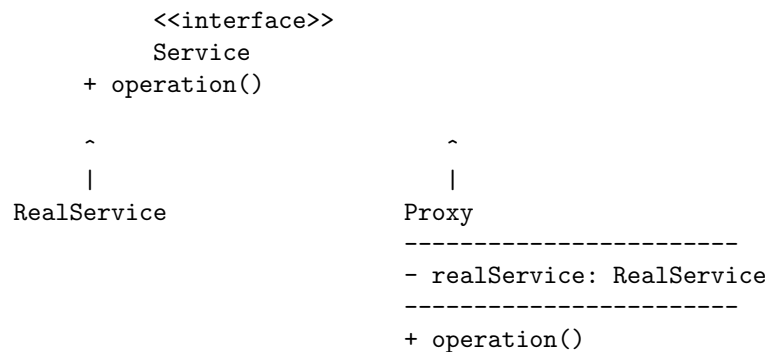
Vysvetlenie:

- Klient komunikuje iba s triedou `OrderFacade`.
- Fasáda interne používa služby `OrderService`, `PaymentService`, `InventoryService` a `NotificationService`.
- Klient nemusí poznať detaily subsystemu.

Facade znižuje väzbu medzi klientom a zložitým subsystémom.

6.12 Proxy – UML opis

Proxy zastupuje iný objekt a kontroluje prístup k nemu.

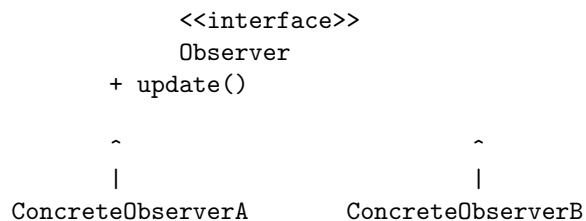


Vysvetlenie:

- **Service** je spoločné rozhranie.
- **RealService** je skutočný objekt.
- **Proxy** má rovnaké rozhranie a obsahuje referenciu na **RealService**.
- **Proxy** môže pred delegovaním vykonať kontrolu práv, lazy loading, logovanie alebo vzdialené volanie.

6.13 Observer – UML opis

Observer umožňuje automaticky informovať závislé objekty o zmene stavu.



Subject

```

-----
- observers: List<Observer>
-----
+ attach(o: Observer)
+ detach(o: Observer)
+ notify()
+ setState(state)
+ getState()

```

Vysvetlenie:

- Subject udržiava zoznam pozorovateľov.
- Pozorovatelia implementujú rozhranie **Observer**.
- Pri zmene stavu subjekt zavolá **notify()**.
- **notify()** zavolá **update()** na všetkých registrovaných pozorovateľoch.

Observer sa často používa v event systémoch a používateľských rozhraniach.

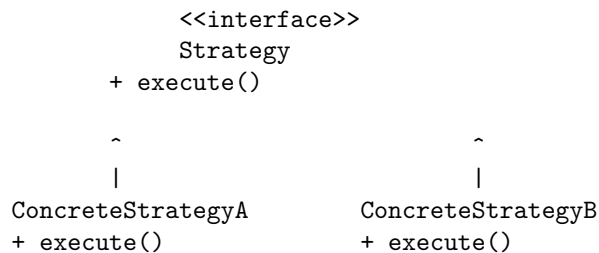
6.14 Strategy – UML opis

Strategy umožňuje vymieňať algoritmus bez zmeny klienta.

Context

```
-----  
- strategy: Strategy  
-----
```

```
+ setStrategy(s: Strategy)  
+ executeStrategy()
```



Vysvetlenie:

- Strategy definuje rozhranie algoritmu.
- ConcreteStrategyA a ConcreteStrategyB implementujú rôzne varianty algoritmu.
- Context používa stratégiu cez rozhranie.
- Stratégiu možno meniť bez zmeny triedy Context.

Strategy pomáha odstrániť rozsiahle vetvenie typu **if** alebo **switch**, keď sa správanie mení podľa zvoleného algoritmu.

6.15 Template Method – UML opis

Template Method definuje kostru algoritmu v abstraktnej triede.

```
AbstractProcessor
-----
+ process()
# step1()
# step2()
# step3()

ConcreteProcessorA
-----
# step1()
# step2()

ConcreteProcessorB
-----
# step1()
# step2()
```

Vysvetlenie:

- `process()` je template method.
- Táto metóda definuje poradie krokov algoritmu.
- Niektoré kroky sú abstraktné a implementujú ich podtriedy.
- Spoločná štruktúra algoritmu zostáva v abstraktnej triede.

Template Method je vhodný, keď majú viaceré algoritmy rovnakú kostru, ale líšia sa v detailoch.

6.16 Visitor – UML opis

Visitor umožňuje pridávať nové operácie nad objektovou štruktúrou bez zmeny tried prvkov.

```
    <<interface>>
    Element
    + accept(v: Visitor)

    ^
    |
ConcreteElementA    ConcreteElementB
+ accept(v: Visitor) + accept(v: Visitor)
```

```

        <<interface>>
        Visitor
        + visitA(e: ConcreteElementA)
        + visitB(e: ConcreteElementB)

        ~
        |
ConcreteVisitor
+ visitA(e: ConcreteElementA)
+ visitB(e: ConcreteElementB)

```

Vysvetlenie:

- Prvky objektovej štruktúry implementujú **Element**.
- Každý prvok má metódu **accept()**.
- Visitor obsahuje metódy pre jednotlivé typy prvkov.
- Novú operáciu možno pridať vytvorením novej triedy visitoru.

Nevýhodou je, že ak často pribúdajú nové typy prvkov, treba meniť rozhranie **Visitor** aj všetky jeho implementácie.

6.17 Súvis s refaktoringom a code smells

Návrhové vzory často súvisia s refaktoringom. Code smell je povrchový znak, ktorý môže naznačovať hlbší problém v návrhu. Nie je to priamo chyba, ale môže signalizovať technický dlh.

Príklady code smells:

- dlhá trieda,
- príliš veľa argumentov v metóde,
- veľký **switch** príkaz,
- paralelné dedičné hierarchie,
- opakujúci sa kód,
- príliš veľa komentárov.

Refaktoring je proces úpravy vnútornej štruktúry kódu bez zmeny jeho vonkajšieho správania. Niekedy môže refaktoring viesť k použitiu návrhového vzoru, napríklad:

- nahradenie **switch** príkazu vzorom **Strategy**,
- oddelenie zložitého vytvárania objektu pomocou **Builder**,
- zjednodušenie subsystému pomocou **Facade**,
- pridávanie správania pomocou **Decorator**.

6.18 Zhrnutie

Návrhové vzory sú opakovateľné riešenia častých problémov v návrhu softvéru. Nie sú o kopírovaní kódu, ale o typických štruktúrach tried, objektov a zodpovedností.

Vytvárajúce vzory riešia vytváranie objektov. Patria sem napríklad Factory Method, Abstract Factory, Builder, Object Pool, Prototype a Singleton.

Štrukturálne vzory riešia skladanie tried a objektov do väčších štruktúr. Patria sem Decorator, Composite, Facade, Adapter, Proxy a Flyweight.

Vzory správania riešia komunikáciu medzi objektmi a rozdelenie zodpovedností pri vykonávaní správania. Patria sem Iterator, Observer, Strategy, Template Method, Null Object, Memento a Visitor.

Na skúške je vhodné vedieť nielen vymenovať základné vzory, ale aj vysvetliť, aký problém riešia, kedy sa používajú a ako vyzerá ich základná UML štruktúra.

7 Immutable dátové štruktúry a testovateľnosť OO kódu

7.1 Immutable dátové štruktúry

Immutable dátová štruktúra je dátová štruktúra, ktorej stav sa po vytvorení nemení. Operácie, ktoré by pri mutable štruktúre menili existujúci objekt, pri immutable štruktúre vytvárajú novú verziu štruktúry.

Napríklad pri mutable zozname operácia `add(x)` zmení existujúci zoznam. Pri immutable zozname operácia `add(x)` vráti nový zoznam, ktorý obsahuje nový prvok, pričom pôvodný zoznam zostáva nezmenený.

Mutable prístup: $s.add(x)$ mení s

Immutable prístup: $s_2 = s.add(x)$ vytvára novú hodnotu

Immutable objekt teda reprezentuje hodnotu, nie meniteľnú entitu. Dve hodnoty s rovnakým obsahom možno chápať ako rovnaké bez ohľadu na ich identitu v pamäti.

7.2 Základné pojmy

7.2.1 Hodnota

Hodnota je dátový objekt, ktorý je po zverejnení nemeniteľný. Môže byť dočasne mutable počas fázy vytvárania, ale po tom, čo je sprístupnený okolitému kódu, už sa nesmie meniť.

Vlastnosti hodnoty:

- je nezávislá od miesta v pamäti,
- po zverejnení sa nemení,

- možno ju bezpečne zdieľať,
- možno ju použiť ako vstup alebo výstup čistej funkcie.

7.2.2 Čistá funkcia

Čistá funkcia je funkcia, ktorá:

- má ako vstup hodnotu,
- vracia hodnotu,
- nemá vedľajšie efekty,
- nemení globálny stav,
- pri rovnakých vstupoch vždy vráti rovnaký výsledok.

Príklad:

```
newList = oldList.add(x)
```

Táto operácia nemení `oldList`, ale vytvorí novú hodnotu `newList`.

7.2.3 Perzistentná dátová štruktúra

Perzistentná dátová štruktúra je immutable dátová štruktúra, ktorá zachováva prístup k predchádzajúcim verziám. Nová verzia vznikne z pôvodnej, ale pôvodná verzia zostáva použiteľná.

Namiesto prepísania starého stavu sa čas modeluje ako postupnosť hodnôt:

$$state_0 \rightarrow state_1 \rightarrow state_2 \rightarrow \dots$$

Nový stav vzniká aplikovaním čistej funkcie na starý stav:

$$state_{i+1} = f(state_i)$$

7.3 Interface immutable tried

Interface immutable triedy musí byť navrhnutý tak, aby neumožňoval meniť existujúcu inštanciu. To znamená, že neobsahuje mutujúce operácie typu `set`, `add` alebo `remove`, ktoré by menili aktuálny objekt. Namiesto toho obsahuje operácie, ktoré vracajú novú verziu objektu.

Príklad interface immutable zoznamu:

```
interface ImmutableList<T> {
    int size();
    T get(int index);

    ImmutableList<T> add(T value);
    ImmutableList<T> set(int index, T value);
    ImmutableList<T> remove(int index);
}
```

Dôležité je, že:

- `add` nemení aktuálny zoznam,
- `set` nemení aktuálny zoznam,
- `remove` nemení aktuálny zoznam,
- každá z týchto operácií vracia nový `ImmutableList`.

Pri immutable triedach sa typicky dodržiavajú tieto pravidlá:

- všetky atribúty sú súkromné,
- atribúty sa nastavujú iba pri konštrukcii,
- objekt neposkytuje mutujúce metódy,
- vnútorné mutable objekty sa nezverejňujú priamo,
- ak trieda obsahuje referencie na kolekcie, tie musia byť tiež immutable alebo musia byť bezpečne kopírované,
- operácie vracajú nové inštancie.

7.4 Implementácia immutable verzií základných štruktúr

7.4.1 Immutable záznam alebo štruktúra

Najjednoduchší prípad je immutable objekt s niekoľkými atribútmi.

Príklad:

```
class Person {
    private final String name;
    private final int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    Person withAge(int newAge) {
        return new Person(this.name, newAge);
    }
}
```

Metóda `withAge` nemení pôvodný objekt, ale vytvorí nový objekt s upravenou hodnotou veku.

7.4.2 Immutable zásobník

Immutable zásobník možno implementovať ako jednosmerne viazaný zoznam.

```
class Stack<T> {
    private final T head;
    private final Stack<T> tail;

    Stack<T> push(T value) {
        return new Stack<T>(value, this);
    }

    Stack<T> pop() {
        return this.tail;
    }

    T top() {
        return this.head;
    }
}
```

Operácia **push** vytvorí nový uzol, ktorý ukazuje na pôvodný zásobník. Pôvodný zásobník zostáva nezmenený.

Výhodou je štruktúrálny zdieľanie:

nový zásobník zdieľa pôvodný zásobník ako svoj chvost

7.4.3 Immutable zoznam

Jednoduchý immutable zoznam možno implementovať ako linked list. Operácia pridania na začiatok je lacná, pretože vytvorí iba nový uzol.

Pri náhodnom prístupe však linked list nie je vhodný. Preto sa v praxi immutable zoznamy alebo vektory často implementujú ako stromy s vysokým vetvením. Pri zmene jedného prvku sa nevytvára celá nová štruktúra, ale iba uzly na ceste k zmenenému prvku. Nezmenené časti sa zdieľajú medzi starou a novou verziou.

7.4.4 Immutable mapa

Immutable mapa môže byť implementovaná napríklad pomocou:

- vyváženého binárneho stromu,
- B-stromu,
- hash trie,
- inej stromovej štruktúry s vysokým vetvením.

Operácia `put(key, value)` vytvorí novú mapu. Nezmenené časti stromu sa zdieľajú so starou mapou.

```
map2 = map1.put("name", "Peter")
```

Po tejto operácii stále existuje `map1` aj `map2`.

7.4.5 Immutable množina

Immutable množina je podobná immutable mape. Prvky sú uložené v stromovej alebo hashovacej štruktúre. Operácie `add` a `remove` vracajú novú množinu.

```
set2 = set1.add(x)
set3 = set2.remove(y)
```

7.4.6 Immutable pole alebo vektor

Pri klasickom poli by zmena jedného prvku vyžadovala skopírovanie celého poľa. To by bolo neefektívne. Preto sa immutable polia často implementujú ako stromy s vysokým vetvením.

Ak zmeníme jeden prvok, vytvoria sa iba nové uzly na ceste od koreňa k danému prvku. Ostatné uzly sa zdieľajú.

Časová zložitosť zmeny jedného prvku je typicky logaritmická, pričom základ logaritmu je vysoký, takže v praxi môže byť zmena pomerne efektívna.

7.5 Štruktúrne zdieľanie

Štruktúrne zdieľanie znamená, že stará a nová verzia dátovej štruktúry zdieľajú tie časti, ktoré sa nezmenili.

Napríklad pri zmene jedného prvku v strome netreba kopírovať celý strom. Stačí vytvoriť nové uzly na ceste k zmenenému prvku a ostatné podstromy použiť z pôvodnej štruktúry.

stará verzia a nová verzia môžu zdieľať väčšinu uzlov

Toto je bezpečné práve preto, že uzly sú immutable.

7.6 Výhody immutable dátových štruktúr

1. Jednoduchšie uvažovanie o programe

Ak sa hodnota nemení, je jednoduchšie pochopiť, čo sa s ňou deje. Nemusíme sledovať, kto ju mohol medzičasom zmeniť.

2. Bezpečné zdieľanie

Immutable objekty možno bezpečne zdieľať medzi viacerými časťami programu. Žiadna časť programu ich nemôže neočakávane zmeniť.

3. Lepšia podpora paralelizmu a concurrency

Pri immutable dátach čítanie nespôsobuje race conditions. Viacero vlákien môže čítať rovnakú hodnotu bez zamykania.

4. Menej potreby lockov

Keďže immutable dáta sa nemenia, netreba chrániť ich čítanie zámkami. Zámok alebo atomická operácia je potrebná najmä pri výmene referencie na aktuálnu verziu.

5. Možnosť uchovávať históriu

Perzistentné dátové štruktúry prirodzene uchovávajú predchádzajúce verzie. To je užitočné napríklad pre undo/redo, audit alebo debugovanie.

6. Jednoduchšie testovanie

Čisté funkcie nad immutable hodnotami sa testujú jednoducho: pre daný vstup očakávame konkrétny výstup. Nie je potrebné pripravovať zložitý globálny stav.

7. Menej skrytých vedľajších efektov

Operácie nad immutable štruktúrami nemenia vstupy, čím znižujú riziko, že zmena na jednom mieste pokazí iné miesto programu.

7.7 Nevýhody immutable dátových štruktúr

1. Režijné náklady

Pri každej zmene vzniká nová verzia štruktúry. Aj keď sa používa štruktúrne zdieľanie, stále vznikajú nové objekty.

2. Vyššia pamäťová náročnosť

Immutable štruktúry môžu spotrebovať viac pamäte, najmä ak sa uchováva veľa historických verzií.

3. Zložitejšia implementácia

Efektívne immutable štruktúry, napríklad hash trie alebo vektorový strom, sú implementačne zložitejšie než jednoduché mutable polia alebo zoznamy.

4. Nižší výkon pri niektorých operáciách

Niektoré operácie môžu byť pomalšie než pri mutable štruktúrach. Napríklad zmena prvku v obyčajnom poli je konštantná, zatiaľ čo pri immutable stro-movej implementácii je typicky logaritmická.

5. Potreba mutable referencie na aktuálny stav

Aj pri immutable dátach často potrebujeme niekde držať referenciu na aktuálnu verziu stavu. Táto referencia je mutable a treba ju spravovať opatrne.

7.8 Testovateľnosť objektovo orientovaného kódu

Testovateľnosť vyjadruje, ako ľahko možno overiť správanie triedy alebo modulu pomocou automatizovaných testov. Objektovo orientovaný kód sa dobre testuje vtedy, keď má trieda jasnú zodpovednosť, explicitné závislosti, malé verejné rozhranie a správa sa deterministicky.

Unit test by mal testovať jednu jednotku kódu, typicky triedu alebo metódu, izolovane od zvyšku systému. Problém vzniká vtedy, keď testovaná trieda priamo komunikuje s databázou, sieťou, súborovým systémom, časom, náhodou alebo globálnym stavom.

7.9 Prvky, ktoré sťažujú testovanie

7.9.1 Skryté závislosti

Trieda sa ťažko testuje, ak si sama vytvára svoje závislosti alebo ich berie z globálneho stavu.

Zlý príklad:

```
class OrderService {
    private PaymentGateway gateway = new RealPaymentGateway();

    void pay(Order order) {
        gateway.pay(order);
    }
}
```

Trieda `OrderService` je pevne naviazaná na `RealPaymentGateway`. V teste ju nemožno jednoducho nahradiť falošnou platobnou bránou.

7.9.2 Používanie globálneho stavu

Globálne premenné, singletony a statické registre zhoršujú testovateľnosť, pretože testy sa môžu navzájom ovplyvňovať.

Príklady:

- globálna konfigurácia,
- globálny logger so stavom,
- singleton databázového spojenia,
- statický cache objekt.

7.9.3 Priamy prístup k externým systémom

Unit testy sa ťažko píše, ak trieda priamo používa:

- databázu,

- súborový systém,
- sieť,
- systémový čas,
- generátor náhodných čísel,
- externé API.

Takéto testy sú pomalé, nestabilné a závislé od prostredia.

7.9.4 Nedeterministické správanie

Kód sa zle testuje, ak výsledok závisí od času, náhody, poradia vlákien alebo aktuálneho stavu okolia.

Príklad:

```
boolean isExpired(Session s) {
    return s.expiresAt() < System.currentTimeMillis();
}
```

Test závisí od reálneho času. Lepšie je zaviesť abstrakciu `Clock`.

7.9.5 Príliš veľké triedy

Trieda s veľkým množstvom zodpovedností sa testuje ťažko, pretože treba pripraviť veľa vstupov, závislostí a stavov. Porušuje to princíp jednej zodpovednosti.

7.9.6 Príliš veľké metódy

Dlhé metódy často obsahujú veľa vetvení, vedľajších efektov a špeciálnych prípadov. Ťažko sa pre ne navrhujú testovacie scenáre.

7.9.7 Silné prepojenie tried

Ak trieda závisí od mnohých konkrétnych tried, je ťažké ju testovať izolovane. Každý test musí vytvoriť celý graf objektov.

7.9.8 Statické metódy s vedľajšími efektmi

Statické metódy samy osebe nemusia byť problém, ak sú čisté. Problém nastáva, ak statická metóda pracuje s globálnym stavom, súborom, sieťou alebo databázou. Takúto závislosť je ťažké nahradiť v teste.

7.9.9 Privátna logika namiesto testovateľného návrhu

Privátne metódy sa zvyčajne netestujú priamo. Ak je veľká časť dôležitej logiky ukrytá v zložitej privátnej metóde, môže to signalizovať, že treba extrahovať novú triedu alebo funkciu.

7.9.10 Konštruktory s veľkou logikou

Konštruktor by mal objekt inicializovať, nie vykonávať zložité operácie, pripájať sa k databáze alebo volať externé služby. Takýto objekt je ťažké vytvoriť v teste.

7.10 Ako navrhnuť objekt tak, aby sa dal jednoduchšie testovať

7.10.1 Explicitné závislosti

Závislosti triedy majú byť explicitné. Ideálne sa odovzdávajú cez konštruktor alebo metódu.

Lepší príklad:

```
class OrderService {
    private final PaymentGateway gateway;

    OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    void pay(Order order) {
        gateway.pay(order);
    }
}
```

V produkcii použijeme reálnu implementáciu. V teste použijeme fake alebo mock.

7.10.2 Programovanie voči rozhraniu

Trieda by nemala závisieť od konkrétnej implementácie, ale od rozhrania.

```
interface PaymentGateway {
    PaymentResult pay(Order order);
}
```

Výhoda je, že v teste možno použiť náhradnú implementáciu.

7.10.3 Dependency Injection

Dependency Injection znamená, že závislosti sa triede dodajú zvonka, namiesto toho, aby si ich trieda vytvárala sama.

Typy dependency injection:

- constructor injection,
- setter injection,

- method injection.

Najčastejšie je vhodná constructor injection, pretože objekt po vytvorení dostane všetky potrebné závislosti a môže byť v konzistentnom stave.

7.10.4 Oddelenie čistej logiky od efektov

Čistú doménovú logiku je vhodné oddeliť od vstupno-výstupných operácií.
Zle:

```
void processOrder(int orderId) {
    Order order = database.load(orderId);
    order.calculatePrice();
    database.save(order);
    email.sendConfirmation(order);
}
```

Lepšie je oddeliť:

- načítanie dát,
- čistý výpočet,
- uloženie dát,
- odoslanie notifikácie.

Čistý výpočet možno potom testovať jednoducho a rýchlo.

7.10.5 Malé triedy s jasnou zodpovednosťou

Trieda by mala mať jednu hlavnú zodpovednosť. Ak trieda rieši objednávky, platby, e-maily aj exporty, treba ju rozdeliť.

7.10.6 Immutable objekty

Immutable objekty sa testujú jednoduchšie, pretože ich stav sa nemení. Test nemusí riešiť, kto objekt medzičasom zmenil.

Príklad:

```
Money m1 = new Money(10, "EUR");
Money m2 = m1.add(new Money(5, "EUR"));

assert m1.amount() == 10;
assert m2.amount() == 15;
```

7.10.7 Abstrakcia času a náhody

Čas a náhoda by sa nemali volať priamo v doménovej logike.

Namiesto:

```
System.currentTimeMillis()  
Math.random()
```

je lepšie použiť:

```
Clock  
RandomGenerator  
IdGenerator
```

V teste možno potom dodať deterministickú implementáciu.

7.10.8 Oddelenie vytvárania objektov od ich používania

Ak trieda vytvára zložité objekty, môže byť vhodné použiť factory alebo builder. Trieda potom nemusí poznať konkrétne implementácie a v teste možno factory nahradiť testovacou verziou.

7.11 Ako sa pri unit testoch vysporiadať so závislosťami triedy

Pri unit testovaní chceme testovať správanie testovanej triedy, nie správanie všetkých jej závislostí. Preto závislosti často nahrádzame testovacími dvojníkmi, po anglicky *test doubles*.

7.11.1 Dummy

Dummy objekt sa odovzdá iba preto, aby bol splnený podpis metódy alebo konštruktora. Test ho reálne nepoužíva.

7.11.2 Stub

Stub vracia pripravené odpovede. Používa sa, keď testovaná trieda potrebuje od závislosti nejaké dáta.

Príklad:

```
class StubExchangeRateProvider implements ExchangeRateProvider {  
    BigDecimal rate(String from, String to) {  
        return new BigDecimal("1.10");  
    }  
}
```

7.11.3 Fake

Fake je jednoduchá funkčná implementácia závislosti. Typickým príkladom je in-memory databáza alebo in-memory repository.

Príklad:

```
class InMemoryUserRepository implements UserRepository {
    private Map<ID, User> users = new HashMap<>();

    void save(User user) {
        users.put(user.id(), user);
    }

    User find(ID id) {
        return users.get(id);
    }
}
```

Fake je vhodný, keď chceme testovať realistickejšie správanie bez použitia skutočnej databázy.

7.11.4 Mock

Mock overuje, či testovaná trieda správne komunikuje so závislosťou. Používa sa napríklad na kontrolu, že bola zavolaná určitá metóda.

Príklad:

```
verify(emailSender).sendConfirmation(order);
```

Mocky sú užitočné pri testovaní interakcií, ale nemali by sa používať nadmerne. Príliš veľa mockov môže znamenať, že test overuje implementačné detaily namiesto správania.

7.11.5 Spy

Spy je objekt, ktorý sa správa ako reálna závislosť, ale zároveň zaznamenáva, ako bol použitý. Je to kombinácia reálnej implementácie a mocku.

7.12 Prístupy k závislostiam v unit testoch

1. Použiť skutočnú závislosť

Vhodné, ak je závislosť jednoduchá, rýchla, deterministická a nemá vedľajšie efekty.

2. Použiť stub

Vhodné, ak potrebujeme vrátiť konkrétne dáta.

3. Použití fake

Vhodné, ak potrebujeme jednoduchú funkčnú náhradu, napríklad in-memory repository.

4. Použití mock

Vhodné, ak chceme overiť interakciu, napríklad že sa odoslal e-mail alebo zavolať platobná brána.

5. Oddeliť závislosť pomocou interface

Testovaná trieda závisí od rozhrania, nie od konkrétnej implementácie.

6. Presunúť externé efekty na hranicu systému

Doménová logika by mala byť čo najčistejšia. Databáza, sieť, čas a súbory by mali byť v okrajových vrstvách systému.

7.13 Príklad testovateľného návrhu

Zle testovateľný návrh:

```
class InvoiceService {
    void createInvoice(Order order) {
        Database db = new Database();
        PdfGenerator pdf = new PdfGenerator();
        EmailSender email = new EmailSender();

        Invoice invoice = calculate(order);
        db.save(invoice);
        email.send(pdf.generate(invoice));
    }
}
```

Problémy:

- trieda si sama vytvára závislosti,
- priamo používa databázu,
- priamo generuje PDF,
- priamo posiela e-mail,
- ťažko sa testuje izolovane.

Lepší návrh:

```
class InvoiceService {
    private final InvoiceRepository repository;
    private final PdfGenerator pdfGenerator;
```

```

private final EmailSender emailSender;

InvoiceService(
    InvoiceRepository repository,
    PdfGenerator pdfGenerator,
    EmailSender emailSender
) {
    this.repository = repository;
    this.pdfGenerator = pdfGenerator;
    this.emailSender = emailSender;
}

Invoice createInvoice(Order order) {
    Invoice invoice = calculate(order);
    repository.save(invoice);
    emailSender.send(pdfGenerator.generate(invoice));
    return invoice;
}
}

```

V teste možno použiť:

- InMemoryInvoiceRepository,
- FakePdfGenerator,
- MockEmailSender.

7.14 Súvis immutable dát a testovateľnosti

Immutable dátové štruktúry a testovateľnosť spolu úzko súvisia. Immutable objekty znižujú počet možných stavov systému, zabraňujú nečakaným zmenám a uľahčujú písanie deterministických testov.

Čisté funkcie nad immutable hodnotami majú jednoduchý testovací model:

$$\text{vstup} \rightarrow \text{funkcia} \rightarrow \text{očakávaný výstup}$$

Nie je potrebné kontrolovať skrytý stav objektu ani vedľajšie efekty. To znižuje zložitosť unit testov.

7.15 Zhrnutie

Immutable dátové štruktúry sú štruktúry, ktoré sa po vytvorení nemenia. Operácie nad nimi neprepisujú existujúci objekt, ale vracajú novú verziu. Efektívna implementácia využíva štruktúrne zdieľanie, pri ktorom stará a nová verzia zdieľajú nezmenené časti štruktúry.

Výhodami immutable dát sú jednoduchšie uvažovanie, bezpečné zdieľanie, lepšia podpora concurrency, menej potreby zámkov, možnosť uchovávať históriu

a jednoduchšie testovanie. Nevýhodami sú režijné náklady, vyššia pamäťová náročnosť a zložitejšia implementácia efektívnych dátových štruktúr.

Testovanie OO kódu sťažujú najmä skryté závislosti, globálny stav, singletony, statické metódy s vedľajšími efektmi, priame volania databázy alebo siete, nedeterministické správanie, veľké triedy, veľké metódy a silné prepojenie tried.

Testovateľný objekt má jasnú zodpovednosť, explicitné závislosti, závisí od rozhraní, používa dependency injection, oddeľuje čistú logiku od efektov a minimalizuje mutable stav. Pri unit testoch sa závislosti nahrádzajú testovacími dvojníkmi, napríklad dummy objektmi, stubmi, fake implementáciami, mockmi alebo spy objektmi.

8 Activity diagramy, sequence diagramy, communication diagramy a state machines

8.1 Úvod

UML poskytuje viacero dynamických diagramov, ktoré opisujú správanie softvérového systému. Kým class diagram opisuje statickú štruktúru systému, activity diagramy, sequence diagramy, communication diagramy a state machine diagramy opisujú, ako sa systém správa v čase.

Tieto diagramy sa používajú najmä na:

- opis algoritmov a pracovných postupov,
- opis scenárov prípadov použitia,
- opis spolupráce objektov,
- opis poradia správ medzi objektmi,
- opis stavovo závislého správania objektov,
- podporu návrhu, implementácie a testovania.

8.2 Activity diagram

Activity diagram je behaviorálny UML diagram, ktorý slúži na vizuálny opis algoritmov, workflow alebo procesov. Je vhodný najmä vtedy, keď chceme ukázať postupnosť činností, rozhodovanie, paralelizmus alebo rozdelenie zodpovedností.

8.2.1 Základné elementy activity diagramu

Medzi základné elementy activity diagramu patria:

1. Initial node

Počiatočný uzol označuje začiatok aktivity. Zobrazuje sa ako plný čierny kruh.

2. **Action**

Akcia predstavuje konkrétny krok alebo jednotku práce. Môže ísť napríklad o výpočet, validáciu údajov, zobrazenie informácie alebo volanie služby.

3. **Control flow**

Control flow určuje poradie vykonávania akcií. Zobrazuje sa šípkou medzi uzlami.

4. **Decision node**

Rozhodovací uzol umožňuje vetvenie toku podľa podmienky. Zobrazuje sa ako kosoštvorec. Výstupné hrany majú podmienky, napríklad `[valid]` a `[invalid]`.

5. **Merge node**

Merge node zlučuje alternatívne vetvy späť do jedného toku.

6. **Fork node**

Fork node rozdeľuje jeden tok na viacero paralelných tokov.

7. **Join node**

Join node synchronizuje viacero paralelných tokov späť do jedného toku.

8. **Activity final node**

Koncový uzol označuje ukončenie celej activity. Zobrazuje sa ako kruh s bodkou vnútri.

9. **Object node**

Object node zobrazuje objekt alebo dáta, ktoré sú vstupom alebo výstupom akcie.

10. **Object flow**

Object flow ukazuje tok objektov alebo dát medzi akciami.

11. **Partition, teda swimlane**

Swimlane je logické rozdelenie diagramu podľa zodpovedností. Môže reprezentovať aktéra, organizačnú rolu, komponent alebo systém.

8.2.2 Použitie activity diagramu

Activity diagram sa používa na:

- opis biznis procesov,
- detailný opis use case scenárov,
- opis algoritmov,

- opis rozhodovania a alternatívnych tokov,
- opis paralelných činností,
- odvodenie testovacích scenárov.

Príklad použitia: pri use case *Checkout* môžeme activity diagramom opísať kroky ako kontrola košíka, zadanie kontaktných údajov, výber dopravy, výber platby, registrácia objednávky a odoslanie potvrdenia.

8.3 Sequence diagram

Sequence diagram je interakčný UML diagram. Opisuje komunikáciu medzi objektmi alebo komponentmi v čase. Hlavný dôraz kladie na poradie správ.

8.3.1 Základné elementy sequence diagramu

Medzi základné elementy sequence diagramu patria:

1. Lifeline

Lifeline reprezentuje účastníka interakcie, napríklad objekt, aktéra, službu, komponent alebo databázu. Zobrazuje sa ako obdĺžnik s prerušovanou zvislou čiarou.

2. Message

Správa reprezentuje komunikáciu medzi účastníkmi. Typicky ide o volanie metódy, odoslanie príkazu alebo návrat hodnoty.

3. Synchronous message

Synchronná správa znamená, že odosielateľ čaká na dokončenie volania.

4. Asynchronous message

Asynchronná správa znamená, že odosielateľ nečaká na okamžitú odpoveď.

5. Return message

Návratová správa zobrazuje návrat výsledku späť od volanej operácie.

6. Activation

Aktivácia ukazuje čas, počas ktorého objekt vykonáva operáciu.

7. Creation message

Správa, ktorá vytvára nový objekt.

8. Destruction

Zničenie objektu sa označuje krížikom na konci lifeline.

9. Combined fragment

Combined fragment opisuje štruktúrované riadenie toku. Najčastejšie:

- **alt** – alternatíva,
- **opt** – voliteľné správanie,
- **loop** – cyklus,
- **par** – paralelné správanie.

8.3.2 Použitie sequence diagramu

Sequence diagram sa používa na:

- opis realizácie use case,
- návrh komunikácie medzi objektmi,
- návrh rozhraní medzi komponentmi,
- opis volaní medzi vrstvami systému,
- dokumentáciu API scenárov,
- hľadanie chýb v poradí volaní,
- odvodenie integračných testov.

Príklad sequence diagramu pre objednávku:

```
Customer -> WebShop: checkout()
WebShop -> Cart: getItems()
WebShop -> PaymentGateway: pay(amount)
PaymentGateway --> WebShop: paymentResult
WebShop -> OrderService: createOrder()
OrderService --> WebShop: order
WebShop --> Customer: confirmation
```

Tento diagram ukazuje poradie komunikácie medzi zákazníkom, e-shopom, košíkom, platobnou bránou a službou objednávok.

8.4 Communication diagram

Communication diagram, staršie nazývaný collaboration diagram, je tiež interakčný UML diagram. Opisuje komunikáciu medzi objektmi, ale na rozdiel od sequence diagramu kladie väčší dôraz na štruktúru spolupráce než na časovú os.

8.4.1 Základné elementy communication diagramu

Medzi základné elementy communication diagramu patria:

1. **Object alebo participant**

Objekt alebo účastník komunikácie.

2. **Link**

Väzba medzi objektmi, po ktorej môžu byť posielané správy.

3. **Message**

Správa posielaná medzi objektmi.

4. **Sequence number**

Číslo správy určuje poradie volaní. Napríklad:

- 1: checkout(),
- 1.1: getItem(),
- 1.2: pay().

8.4.2 Použitie communication diagramu

Communication diagram sa používa na:

- opis spolupráce objektov,
- zdôraznenie väzieb medzi objektmi,
- návrh objektovej štruktúry pri realizácii scenára,
- jednoduchšiu vizualizáciu komunikácie v menších modeloch.

Sequence diagram a communication diagram často opisujú tú istú interakciu.

Rozdiel je v dôraze:

- sequence diagram zdôrazňuje časové poradie,
- communication diagram zdôrazňuje väzby a spoluprácu objektov.

Príklad communication diagramu:

```
customer: Customer
webShop: WebShop
cart: Cart
paymentGateway: PaymentGateway
orderService: OrderService
```

```
1: checkout()
1.1: getItem()
1.2: pay(amount)
1.3: createOrder()
1.4: showConfirmation()
```

8.5 State machine diagram

State machine diagram, teda stavový diagram, je behaviorálny UML diagram, ktorý opisuje stavovo závislé správanie objektu alebo systému. Ukazuje, v akých stavoch sa objekt môže nachádzať a aké udalosti spôsobujú prechody medzi stavmi.

8.5.1 Základné elementy state machine diagramu

Medzi základné elementy state machine diagramu patria:

1. **State**

Stav reprezentuje situáciu, v ktorej sa objekt nachádza určitý čas. Objekt v danom stave môže čakať na udalosť alebo vykonávať určitú činnosť.

2. **Initial state**

Počiatkový stav označuje začiatok životného cyklu objektu.

3. **Final state**

Koncový stav označuje ukončenie životného cyklu objektu alebo procesu.

4. **Transition**

Prechod určuje zmenu zo zdrojového stavu do cieľového stavu.

5. **Event**

Udalosť spúšťa prechod medzi stavmi. Môže ísť o volanie metódy, prijatie správy, časovú udalosť alebo zmenu podmienky.

6. **Guard condition**

Guard je podmienka, ktorá musí byť splnená, aby sa prechod uskutočnil. Zapisuje sa do hranatých zátvoriek.

7. **Action**

Akcia je činnosť vykonaná počas prechodu.

8. **Entry action**

Akcia vykonaná pri vstupe do stavu.

9. **Exit action**

Akcia vykonaná pri opustení stavu.

10. **Do activity**

Činnosť vykonávaná počas zotrvania v stave.

8.5.2 Použitie state machine diagramu

State machine diagram sa používa na:

- opis životného cyklu objektu,
- opis stavovo závislého správania,
- modelovanie protokolov,
- modelovanie workflow s významnými stavmi,
- návrh riadiacej logiky,
- odvodenie testov pre prechody medzi stavmi.

Príklad stavov objednávky:

Created -> Paid -> Shipped -> Delivered
Created -> Cancelled
Paid -> Refunded

Takýto diagram ukazuje, že objednávka nemôže prejsť napríklad priamo zo stavu **Created** do **Delivered**, ak to model nepovoľuje.

8.6 Porovnanie diagramov

Diagram	Hlavné použitie
Activity diagram	Opis workflow, algoritmov, use case scenárov, rozhodovania a paralelizmu.
Sequence diagram	Opis časového poradia správ medzi objektmi alebo komponentmi.
Communication diagram	Opis spolupráce objektov so zameraním na väzby medzi nimi.
State machine diagram	Opis stavov objektu a prechodov medzi nimi.

Table 3: Porovnanie dynamických UML diagramov

8.7 Využitie diagramov pri popise softvéru

Tieto diagramy sa používajú v rôznych fázach vývoja softvéru.

8.7.1 Analýza požiadaviek

V analýze požiadaviek pomáhajú detailnejšie opísať funkčné požiadavky. Activity diagram môže rozpracovať scenár use case. State machine diagram môže opísať životný cyklus významnej doménovej entity.

Príklady:

- activity diagram pre checkout,
- state machine diagram pre objednávku,
- sequence diagram pre reset hesla.

8.7.2 Architektúra a návrh

V návrhu sa sequence a communication diagramy používajú na určenie, ktoré objekty alebo komponenty spolupracujú pri realizácii funkcionality.

Príklady:

- sequence diagram medzi controllerom, service triedou a repository,
- communication diagram spolupráce objektov v doménovej vrstve,
- state machine diagram objektu, ktorý má komplikovaný životný cyklus.

8.7.3 Implementácia

Pri implementácii diagramy pomáhajú vývojárom pochopiť očakávané správanie. Sequence diagram môže slúžiť ako návod na implementáciu poradia volaní. State machine diagram môže byť priamo mapovaný na stavový automat v kóde.

8.7.4 Testovanie

Diagramy pomáhajú odvodiť testovacie scenáre:

- z activity diagramu možno odvodiť testy pre hlavné, alternatívne a chybové toky,
- zo sequence diagramu možno odvodiť integračné testy komunikácie,
- zo state machine diagramu možno odvodiť testy povolených a nepovolených prechodov,
- z communication diagramu možno overiť správne väzby medzi objektmi.

8.8 Návrhový vzor vysvetlený pomocou activity diagramu: Template Method

8.8.1 Podstata vzoru Template Method

Template Method je návrhový vzor správania. Používa sa vtedy, keď viacero algoritmov má rovnakú základnú štruktúru, ale líšia sa v niektorých krokoch.

Základná idea je:

- abstraktná trieda definuje kostru algoritmu,
- niektoré kroky algoritmu sú pevne dané,
- niektoré kroky sú abstraktné alebo prepisovateľné,
- podtriedy implementujú konkrétne varianty týchto krokov.

Príklad: spracovanie dokumentu môže mať vždy rovnaký všeobecný postup:

1. načítaj dokument,
2. validuj dokument,
3. spracuj obsah,
4. ulož výsledok.

Rozdiel je v tom, či spracúvame PDF, XML alebo CSV dokument. Kostra algoritmu je rovnaká, ale niektoré kroky sa líšia.

8.8.2 Class diagram vzoru Template Method

AbstractDocumentProcessor

```
-----  
+ process()  
# loadDocument()  
# validateDocument()  
# processContent()  
# saveResult()
```

PdfProcessor

```
-----  
# loadDocument()  
# processContent()
```

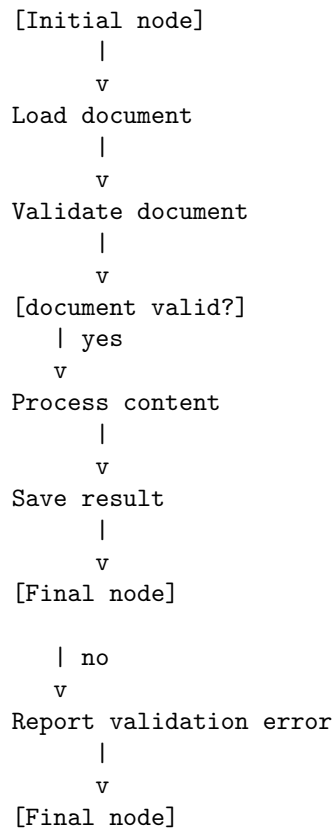
XmlProcessor

```
-----  
# loadDocument()  
# validateDocument()  
# processContent()
```

Metóda `process()` je template method. Definuje poradie krokov. Podtriedy prepisujú konkrétne kroky.

8.8.3 Activity diagram ako kľúčové vysvetlenie

Activity diagram je pri tomto vzore kľúčový, pretože Template Method je o pevne danej štruktúre algoritmu. Diagram ukazuje, ktoré časti algoritmu sú spoločné a ktoré sú variabilné.



Tento activity diagram opisuje všeobecný algoritmus v abstraktnej triede. Konkrétne triedy nemenia poradie krokov, ale implementujú vybrané akcie.

8.8.4 Mapovanie activity diagramu na kód

Template method môže vyzeráť napríklad takto:

```
abstract class DocumentProcessor {

    public final void process() {
        Document document = loadDocument();

        if (!validateDocument(document)) {
            reportValidationError(document);
        }
    }
}
```

```

        return;
    }

    Result result = processContent(document);
    saveResult(result);
}

protected abstract Document loadDocument();

protected boolean validateDocument(Document document) {
    return true;
}

protected abstract Result processContent(Document document);

protected void saveResult(Result result) {
    // default implementation
}

protected void reportValidationError(Document document) {
    // default implementation
}
}

```

Trieda `PdfProcessor` môže implementovať načítanie PDF a spracovanie PDF obsahu. Trieda `XmlProcessor` môže implementovať načítanie XML a vlastnú validáciu.

8.8.5 Prečo activity diagram dobre vysvetľuje Template Method

Activity diagram jasne ukazuje:

- pevné poradie krokov algoritmu,
- rozhodovacie body,
- spoločné správanie,
- miesta, kde sa správanie môže meniť v podtriedach.

Preto je activity diagram vhodnejší než samotný class diagram, ak chceme vysvetliť dynamiku vzoru Template Method. Class diagram ukáže štruktúru tried, ale activity diagram ukáže algoritmus, teda hlavnú myšlienku vzoru.

8.9 Alternatívny príklad so state machine: stav objednávky

State machine diagram je vhodný aj na vysvetlenie vzoru State. Tento vzor sa používa vtedy, keď sa objekt správa rôzne podľa svojho aktuálneho stavu.

Príklad stavov objednávky:

```

Created
  -- pay() --> Paid
  -- cancel() --> Cancelled

Paid
  -- ship() --> Shipped
  -- refund() --> Refunded

Shipped
  -- deliver() --> Delivered

Delivered
  -- return() --> Returned

```

Pri vzore State by trieda `Order` delegovala správanie na objekt reprezentujúci aktuálny stav:

```

Order
-----
- state: OrderState
+ pay()
+ cancel()
+ ship()
+ deliver()

<<interface>>
OrderState
-----
+ pay(order)
+ cancel(order)
+ ship(order)
+ deliver(order)

CreatedState
PaidState
ShippedState
DeliveredState
CancelledState

```

Každý stav implementuje iba tie prechody, ktoré sú v danom stave povolené. Nepovolené prechody môžu vyhodíť výnimku alebo byť odmietnuté. State machine diagram je tu kľúčový, pretože presne definuje povolené stavy a prechody.

8.10 Zhrnutie

Activity diagram opisuje workflow, algoritmy, rozhodovanie a paralelné toky. Je vhodný na detailný opis use case scenárov a procesov.

Sequence diagram opisuje komunikáciu objektov v časovom poradí. Je vhodný na opis realizácie use case, návrh interakcií a integračné scenáre.

Communication diagram opisuje tú istú interakciu ako sequence diagram, ale viac zdôrazňuje väzby medzi objektmi než časovú os.

State machine diagram opisuje stavy objektu a prechody medzi nimi. Je vhodný pre objekty, ktorých správanie závisí od aktuálneho stavu.

Návrhový vzor Template Method možno dobre vysvetliť pomocou activity diagramu, pretože jeho podstatou je pevná kostra algoritmu s variabilnými krokmi. State machine je zase vhodná na vysvetlenie stavovo orientovaného návrhu, napríklad pri vzore State alebo pri modelovaní životného cyklu objednávky.

9 Ostatné UML diagramy a UML profily

9.1 Component diagram

Component diagram je štruktúrny UML diagram, ktorý opisuje systém ako súbor komponentov a ich vzájomných závislostí. Používa sa najmä na architektonickej úrovni, keď chceme zachytiť, z akých väčších softvérových častí sa systém skladá a ako tieto časti spolu komunikujú.

9.1.1 Komponent

Komponent predstavuje samostatnú, nahraditeľnú a znovupoužiteľnú časť systému, ktorá poskytuje určité rozhrania a môže vyžadovať iné rozhrania.

Komponentom môže byť napríklad:

- modul aplikácie,
- knižnica,
- služba,
- subsystém,
- mikroslužba,
- databázový modul,
- používateľské rozhranie,
- externý systém.

Komponent sa v UML označuje stereotypom:

```
<<component>>  
PaymentService
```

alebo špeciálnou ikonou komponentu.

9.1.2 Rozhrania komponentov

Komponenty spolu komunikujú cez rozhrania. Rozlišujeme:

- **provided interface** – rozhranie, ktoré komponent poskytuje,
- **required interface** – rozhranie, ktoré komponent potrebuje od iného komponentu.

Provided interface sa často kreslí ako kruh, tzv. lollipop notation. Required interface sa kreslí ako polkruh, tzv. socket notation.

Príklad:

```
<<component>> OrderService
provided: OrderAPI
required: PaymentAPI
required: InventoryAPI
```

To znamená, že komponent **OrderService** poskytuje rozhranie **OrderAPI**, ale pre svoju činnosť potrebuje platobné a skladové API.

9.1.3 Dependency

Dependency vyjadruje, že jeden komponent závisí od druhého. V UML sa zobrazuje prerušovanou šípkou.

Príklad:

```
OrderService ..> PaymentService
```

Tento zápis znamená, že **OrderService** používa služby komponentu **PaymentService**.

9.1.4 Port

Port je komunikačný bod komponentu. Vyjadruje, cez aké miesto komponent komunikuje s okolím.

Port môže byť použitý napríklad vtedy, keď komponent poskytuje alebo vyžaduje viacero rôznych rozhraní a chceme presnejšie ukázať, kadiaľ komunikácia prechádza.

9.1.5 Použitie component diagramu

Component diagram sa používa na:

- opis softvérovej architektúry,
- rozdelenie systému na komponenty,
- opis závislostí medzi komponentmi,
- opis poskytovaných a požadovaných rozhraní,

- návrh mikroslužieb,
- návrh modulárneho systému,
- dokumentáciu integrácií.

Príklad component diagramu e-shopu:

```
<<component>> WebFrontend
<<component>> OrderService
<<component>> PaymentService
<<component>> InventoryService
<<component>> NotificationService
<<component>> Database

WebFrontend ..> OrderService
OrderService ..> PaymentService
OrderService ..> InventoryService
OrderService ..> NotificationService
OrderService ..> Database
```

Takýto diagram poskytuje prehľad o tom, ktoré komponenty v systéme existujú a ktoré komponenty od seba závisia.

9.2 Composite structure diagram

Composite structure diagram je štruktúrny UML diagram, ktorý opisuje vnútornú štruktúru triedy, komponentu alebo iného klasifikátora. Zatiaľ čo class diagram ukazuje vzťahy medzi triedami zvonka, composite structure diagram ukazuje, z akých vnútorných častí sa daný prvok skladá a ako tieto časti spolu komunikujú.

9.2.1 Structured classifier

Structured classifier je prvok, ktorý má vnútornú štruktúru. Môže ísť napríklad o triedu, komponent alebo subsystém.

Príklad:

```
OrderProcessingComponent
```

Tento komponent môže byť vnútorne zložený z častí:

- OrderValidator,
- PaymentHandler,
- InventoryChecker,
- NotificationSender.

9.2.2 Part

Part predstavuje vnútornú časť štruktúrovaného klasifikátora. Je to vlastnosť alebo rola, ktorú hrá určitý objekt vo vnútri väčšieho celku.

Zapísuje sa napríklad:

```
validator: OrderValidator
paymentHandler: PaymentHandler
inventoryChecker: InventoryChecker
```

9.2.3 Connector

Connector spája vnútorné časti medzi sebou alebo s portami na hranici komponentu. Vyjadruje, že medzi časťami existuje komunikačná alebo väzbová cesta.

Príklad:

```
validator -- paymentHandler
paymentHandler -- inventoryChecker
```

9.2.4 Port

Port je miesto, cez ktoré štruktúrovaný prvok komunikuje s okolím. Port môže byť napojený na interné časti.

Príklad:

```
OrderProcessingComponent
port: orderAPI
port: paymentAPI
```

9.2.5 Provided a required interface

Rovnako ako pri component diagrame možno ukázať, ktoré rozhrania daný prvok poskytuje a ktoré vyžaduje.

9.2.6 Použitie composite structure diagramu

Composite structure diagram sa používa na:

- opis vnútornej štruktúry komponentu alebo triedy,
- opis runtime kompozície objektov,
- opis spolupráce častí vo vnútri celku,
- detailnejšie vysvetlenie architektúry komponentu,
- modelovanie portov a konektorov,
- opis zložených objektov.

Príklad:

```
OrderProcessingComponent
-----
parts:
    validator: OrderValidator
    paymentHandler: PaymentHandler
    inventoryChecker: InventoryChecker
    notifier: NotificationSender

connectors:
    validator -- paymentHandler
    paymentHandler -- inventoryChecker
    inventoryChecker -- notifier
```

Tento diagram ukazuje, že komponent `OrderProcessingComponent` nie je čierna skrinka, ale má vnútorné časti, ktoré medzi sebou spolupracujú.

9.3 Package diagram

Package diagram je štruktúrny UML diagram, ktorý slúži na organizáciu modelu alebo softvérového systému do balíkov. Balík predstavuje logické zoskupenie prvkov, napríklad tried, komponentov, use casov alebo iných balíkov.

9.3.1 Package

Package, teda balík, je menný priestor a organizačná jednotka modelu.

Zobrazuje sa ako priečinok:

```
+-----+
| domain |
+-----+
```

Balíky môžu obsahovať napríklad:

- triedy,
- rozhrania,
- komponenty,
- diagramy,
- iné balíky.

9.3.2 Package dependency

Package dependency vyjadruje, že prvky v jednom balíku závisia od prvkov v inom balíku.

Zobrazuje sa prerušovanou šípkou:

```
application ..> domain
infrastructure ..> domain
presentation ..> application
```

9.3.3 Import

Import znamená, že jeden balík používa verejné prvky iného balíka. Zvyčajne sa označuje stereotypom:

```
<<import>>
```

9.3.4 Merge

Package merge znamená zlúčenie obsahu jedného balíka s iným balíkom. Používa sa najmä pri definovaní a rozširovaní modelov.

Označuje sa stereotypom:

```
<<merge>>
```

9.3.5 Použitie package diagramu

Package diagram sa používa na:

- organizáciu veľkého modelu,
- rozdelenie systému na vrstvy,
- opis závislostí medzi balíkmi,
- kontrolu architektonických pravidiel,
- zobrazenie modulárnej štruktúry systému,
- podporu navigácie v dokumentácii.

Príklad vrstvenej architektúry:

```
presentation ..> application
application ..> domain
infrastructure ..> domain
infrastructure ..> application
```

Význam:

- **presentation** obsahuje používateľské rozhranie,

- **application** obsahuje aplikačné služby a use case logiku,
- **domain** obsahuje doménové triedy a pravidlá,
- **infrastructure** obsahuje databázu, súbory, sieť a externé služby.

Package diagram je vhodný najmä pri väčších systémoch, kde by samotné class diagramy boli príliš neprehľadné.

9.4 Deployment diagram

Deployment diagram je štruktúrny UML diagram, ktorý opisuje fyzické alebo virtuálne nasadenie systému. Ukazuje, na akých uzloch bežia jednotlivé artefakty a ako sú tieto uzly prepojené.

9.4.1 Node

Node, teda uzol, predstavuje výpočtový alebo vykonávací zdroj. Môže to byť:

- fyzický server,
- virtuálny server,
- kontajner,
- databázový server,
- klientské zariadenie,
- mobilné zariadenie,
- cloudová služba,
- aplikačný server.

Príklad:

```
<<device>> Mobile phone
<<executionEnvironment>> Docker container
<<node>> Application server
```

9.4.2 Device

Device je špeciálny typ uzla reprezentujúci fyzické zariadenie.

Príklad:

```
<<device>> Web server
<<device>> Database server
<<device>> Client PC
```

9.4.3 Execution environment

Execution environment je prostredie, v ktorom sa vykonáva softvér. Môže ísť napríklad o JVM, aplikačný server, Docker kontajner alebo operačný systém.

Príklad:

```
<<executionEnvironment>> JVM
<<executionEnvironment>> Docker
<<executionEnvironment>> Node.js runtime
```

9.4.4 Artifact

Artifact je fyzický alebo nasaditeľný výsledok vývoja. Môže to byť:

- spustiteľný súbor,
- knižnica,
- JAR súbor,
- WAR súbor,
- Docker image,
- konfiguračný súbor,
- databázová schéma.

Príklad:

```
<<artifact>> webshop.war
<<artifact>> payment-service.jar
<<artifact>> database-schema.sql
```

9.4.5 Communication path

Communication path ukazuje komunikačné spojenie medzi uzlami.

Príklad:

```
Client Browser -- HTTPS -- Web Server
Web Server -- REST -- Application Server
Application Server -- JDBC -- Database Server
```

9.4.6 Deployment relationship

Deployment relationship ukazuje, že určitý artifact je nasadený na konkrétnom uzle.

Príklad:

```
webshop.war deployed on Application Server
database-schema.sql deployed on Database Server
```

9.4.7 Použitie deployment diagramu

Deployment diagram sa používa na:

- opis prevádzkovej architektúry systému,
- opis fyzického alebo virtuálneho nasadenia,
- zobrazenie serverov, klientov a databáz,
- zobrazenie komunikačných protokolov,
- plánovanie infraštruktúry,
- dokumentáciu produkčného, testovacieho alebo vývojového prostredia,
- analýzu bezpečnosti, výkonu a dostupnosti.

Príklad deployment diagramu e-shopu:

```
<<device>> Client device
    <<artifact>> Browser

    HTTPS

<<node>> Web server
    <<artifact>> frontend-app

    REST

<<node>> Application server
    <<artifact>> order-service.jar
    <<artifact>> payment-service.jar

    JDBC

<<node>> Database server
    <<artifact>> ecommerce-db
```

Tento diagram ukazuje, že používateľ komunikuje cez prehliadač s webovým serverom, webový server komunikuje s aplikačným serverom a aplikačný server s databázou.

9.5 Porovnanie diagramov

9.6 UML profily

UML profil je mechanizmus, ktorý umožňuje prispôsobiť UML pre konkrétnu doménu, technológiu, platformu alebo metodiku bez zmeny samotného UML metamodelu.

Diagram	Hlavné použitie
Component diagram	Opis softvérových komponentov, ich rozhraní a závislostí.
Composite structure diagram	Opis vnútornej štruktúry triedy, komponentu alebo subsystému.
Package diagram	Organizácia modelu alebo systému do balíkov a vrstiev.
Deployment diagram	Opis fyzického alebo virtuálneho nasadenia systému.

Table 4: Porovnanie vybraných UML štruktúrnych diagramov

UML je všeobecný modelovací jazyk. V praxi však často potrebujeme modelovať špecifické veci, napríklad:

- databázové tabuľky,
- webové služby,
- enterprise aplikácie,
- embedded systémy,
- bezpečnostné prvky,
- komponenty konkrétnej platformy,
- architektonické vrstvy,
- testovacie prvky.

UML profil umožňuje doplniť UML o špecifickú terminológiu a význam.

9.6.1 Stereotype

Stereotype je najdôležitejší prvok UML profilu. Rozširuje existujúci UML prvok o špecifický význam.

Stereotyp sa zapisuje do dvojitéch lomených zátvoriek:

```
<<controller>>
<<entity>>
<<service>>
<<repository>>
<<component>>
<<device>>
<<executionEnvironment>>
```

Príklad:

```
<<service>>
PaymentService
```

Tento zápis hovorí, že trieda alebo komponent `PaymentService` má v architektúre význam služby.

9.6.2 Tagged value

Tagged value je vlastnosť pridaná stereotypom. Umožňuje doplniť modelový prvok o dodatočné informácie.

Príklad:

```
<<service>>
PaymentService
{protocol = "REST"}
{timeout = "5s"}
```

Tagged values môžu vyjadrovať napríklad:

- protokol,
- verziu,
- autora,
- bezpečnostnú úroveň,
- databázový názov,
- technologickú platformu,
- konfiguračné vlastnosti.

9.6.3 Constraint

Constraint je obmedzenie, ktoré musí modelový prvok spĺňať. Zapisuje sa do zložených zátvoriek.

Príklad:

```
{must be stateless}
{only depends on domain package}
{tableName must be unique}
```

Constraint môže byť zapísaný prirodzeným jazykom alebo formálnejšie, napríklad pomocou OCL.

9.6.4 Profile

Profile je balík stereotypov, tagged values a obmedzení určený pre konkrétny účel.

Príklad profilu pre vrstvenú architektúru:

```
<<controller>>
<<service>>
<<repository>>
<<entity>>
<<valueObject>>
```

Takýto profil umožní v class alebo component diagrame jasne odlíšiť úlohy jednotlivých prvkov.

9.6.5 Použitie UML profilov

UML profily sa používajú na:

- prispôsobenie UML konkrétnej doméne,
- modelovanie platformovo špecifických prvkov,
- zavedenie architektonických pravidiel,
- doplnenie modelu o metadáta,
- zvýšenie čitateľnosti modelu,
- podporu generovania kódu alebo dokumentácie,
- kontrolu konzistencie modelu.

9.6.6 Príklad UML profilu pre webovú aplikáciu

Profile: WebApplicationProfile

Stereotypes:

```
<<controller>> extends Class
<<service>> extends Class
<<repository>> extends Class
<<entity>> extends Class
<<restEndpoint>> extends Interface
```

Tagged values:

```
<<restEndpoint>> {path: String}
<<restEndpoint>> {method: String}
<<repository>> {database: String}
```

Constraints:

```
<<controller>> may depend on <<service>>
<<service>> may depend on <<repository>>
<<repository>> may depend on <<entity>>
<<entity>> should not depend on <<controller>>
```

Tento profil umožňuje modelovať webovú aplikáciu spôsobom, ktorý je bližší reálnej architektúre.

9.7 Zhrnutie

Component diagram opisuje softvérové komponenty, ich poskytované a požadované rozhrania a závislosti. Používa sa najmä na architektonický opis systému.

Composite structure diagram opisuje vnútornú štruktúru triedy, komponentu alebo subsystému. Ukazuje jeho vnútorné časti, porty a konektory.

Package diagram organizuje model alebo systém do balíkov. Používa sa na opis vrstiev, modulov a závislostí medzi časťami systému.

Deployment diagram opisuje fyzické alebo virtuálne nasadenie systému. Ukazuje uzly, artefakty a komunikačné spojenia medzi nimi.

UML profil je mechanizmus rozšírenia UML pre konkrétnu doménu alebo technológiu. Pozostáva najmä zo stereotypov, tagged values a obmedzení. Pomáha prispôbiť UML konkrétnemu projektu, platforme alebo architektúre.

10 Objekty a databázy

10.1 Databáza a DBMS

Databáza je organizovaná kolekcia dát, zvyčajne uložená elektronicky, ktorá umožňuje jednoduché vyhľadávanie, modifikáciu a správu informácií. Databázou môže byť napríklad súbor vo formáte CSV, JSON, XML, tabuľky v relačnej databáze alebo dokumenty v dokumentovej databáze.

DBMS, teda *Database Management System*, je softvér, ktorý spravuje dáta uložené v databáze a umožňuje s nimi pracovať. Príkladom DBMS sú relačné databázové systémy, NoSQL databázy alebo NewSQL databázy.

10.2 Vývoj dátového modelu

Pri návrhu databázy sa typicky rozlišujú tri úrovne dátového modelu:

1. Konceptuálny dátový model

Ide o abstraktný a vysokoúrovňový model dát. Zachytáva entity, abstraktné vzťahy, prípadne atribúty a kardinality. Často je odvodený z doménového modelu a doplnený glosárom.

Konceptuálny model je nezávislý od implementácie a od konkrétneho úložiska dát. Môže byť vyjadrený napríklad ER diagramom, UML class diagramom alebo voľnou formou.

2. Logický dátový model

Logický dátový model je detailnejší. Stále vychádza z biznis potrieb, ale už opisuje entity, atribúty, vzťahy, kardinality a obmedzenia. Môže obsahovať primárne kľúče, cudzie kľúče a pravidlá referenčnej integrity.

Stále však nie je viazaný na konkrétny DBMS.

3. Fyzický dátový model

Fyzický model je databázovo špecifická reprezentácia dát. Opisuje konkrétne tabuľky, stĺpce, indexy, trigger, constraints, uložené procedúry, funkcie a konkrétne dátové typy daného DBMS.

Napríklad fyzický model pre PostgreSQL už používa konkrétne typy ako BIGSERIAL, VARCHAR, TIMESTAMP alebo NUMERIC.

V materiáli je tento proces zobrazený ako prechod od konceptuálneho modelu k logickému a fyzickému modelu, pričom z konceptuálneho modelu sa zároveň odvodzuje objektovo orientovaný model aplikácie. Medzi objektovo orientovaným modelom a relačným dátovým modelom potom typicky stojí ORM.

10.3 Nekompatibilnosť OO systémov a relačných databáz

Objektovo orientované systémy a relačné databázy používajú odlišné modely dát. Tento problém sa často označuje ako *object-relational impedance mismatch*, teda objektovo-relačný nesúlad.

10.3.1 Objektový model

Objektovo orientovaný model pracuje s objektmi. Objekt má:

- identitu,
- stav,
- správanie,
- atribúty,
- odkazy na iné objekty,
- dedičnosť,
- polymorfizmus,
- zapuzdrenie.

Objekty sú navzájom prepojené referenciami. Napríklad objekt `Order` môže obsahovať zoznam objektov `OrderItem`.

10.3.2 Relačný model

Relačný model pracuje s tabuľkami. Dáta sú uložené v riadkoch a stĺpcoch. Vzťahy medzi tabuľkami sa vyjadrujú pomocou primárnych a cudzích kľúčov.

Relačný model je založený najmä na:

- tabuľkách,
- riadkoch,
- stĺpcoch,
- primárnych kľúčoch,
- cudzích kľúčoch,
- referenčnej integrite,
- SQL dotazoch,
- normalizácii.

10.3.3 Základné rozdiely

Nekompatibilita vzniká preto, že objektový a relačný model reprezentujú dáta iným spôsobom.

1. Identita objektov verus primárne kľúče

V OO systéme má objekt identitu danú referenciou alebo identitou objektu v pamäti. V relačnej databáze je identita riadku zvyčajne daná primárnym kľúčom.

Problém nastáva napríklad pri rozhodovaní, či dva objekty s rovnakým ID reprezentujú tú istú databázovú entitu.

2. Referencie verus cudzie kľúče

V OO systéme objekt priamo odkazuje na iný objekt. V relačnej databáze je vzťah reprezentovaný hodnotou cudzieho kľúča.

Príklad:

```
class Order {  
    Customer customer;  
}
```

V databáze:

```
orders(customer_id)
```

3. Dedičnosť

Objektovo orientované jazyky podporujú dedičnosť. Relačný model dedičnosť priamo nepodporuje. Ak máme triedy `Product`, `BookProduct` a `BikeProduct`, musíme rozhodnúť, ako ich uložiť do tabuliek.

Bežné stratégie sú:

- jedna tabuľka pre celú hierarchiu,
- jedna tabuľka pre každú triedu,
- jedna tabuľka pre každú konkrétnu triedu.

4. Asociácie a kolekcie

Objekty môžu obsahovať kolekcie iných objektov, napríklad objednávka obsahuje zoznam položiek. V relačnej databáze sa vzťah 1:N reprezentuje pomocou cudzieho kľúča a vzťah M:N pomocou spojovacej tabuľky.

5. Zapuzdrenie verus tabuľková reprezentácia

Objekt zapuzdruje svoj stav a správanie. Relačná databáza sprístupňuje dáta ako tabuľky. Databáza nepozná metódy objektov, iba dáta a databázové operácie.

6. Granularita

Jeden objekt nemusí zodpovedať jednej tabuľke. Objekt môže byť rozdelený do viacerých tabuliek alebo naopak jedna tabuľka môže obsahovať dáta viacerých typov objektov.

7. Navigácia v dátach

V OO systéme sa často naviguje cez referencie:

```
order.getCustomer().getAddress()
```

V relačnej databáze sa dáta získavajú pomocou dotazov a spojení:

```
SELECT ...  
FROM orders  
JOIN customers ...  
JOIN addresses ...
```

8. Typový systém

Programovací jazyk a databáza môžu mať odlišné typy. Napríklad dátum, čas, enum, peniaze alebo kolekcie sa môžu v jazyku a databáze reprezentovať odlišne.

9. Transakcie a životný cyklus objektov

Objekty existujú v pamäti aplikácie, zatiaľ čo relačné dáta sú perzistentné v databáze. Treba riešiť, kedy sa objekt uloží, aktualizuje, zmaže, načíta alebo odpojí od databázovej session.

10.4 Príklad problému s produktmi

V materiáli je príklad e-shopu, kde rôzne kategórie produktov majú rôzne atribúty. Bicykle majú napríklad veľkosť rámu a kolies, Lego sady majú počet dielikov a pracie prášky majú počet pracích cyklov. Vzniká tak riedka matica atribútov, pretože nie každý produkt používa každý atribút. Naivné riešenie by bolo dať všetky atribúty do tabuľky `Product`, čo by viedlo k mnohým prázdnyhm hodnotám. Materiál preto ukazuje aj relačné riešenia cez špecializované tabuľky alebo EAV model.

10.4.1 Možnosť 1: špecializované tabuľky

Jedna možnosť je mať spoločnú tabuľku `Product` a k nej špecializované tabuľky:

```
Product  
BicycleProduct  
LegoSetProduct  
WashingPowderProduct
```

Výhody:

- jasnejšia štruktúra,
- lepšie typy atribútov,
- jednoduchšie constraints.

Nevýhody:

- horšia flexibilita pri pridávaní nových kategórií,
- viac tabuliek,
- zložitejšie dotazy.

10.4.2 Možnosť 2: EAV model

EAV znamená *entity-attribute-value*. Namiesto samostatných stĺpcov pre každý atribút máme tabuľku atribútov a tabuľku hodnôt.

Príklad:

```
CategoryAttribute(categoryId, name, type)
ProductAttributeValue(productId, attributeId, value)
```

Výhody:

- vysoká flexibilita,
- jednoduché pridávanie nových atribútov,
- vhodné pre veľmi rôznorodé produkty.

Nevýhody:

- slabšie typovanie,
- zložitejšie constraints,
- zložitejšie dotazy,
- horší výkon pri niektorých filtrovaniach,
- časť logiky sa presúva do aplikácie.

10.5 ORM

ORM znamená *Object-Relational Mapping*. Je to technika alebo nástroj, ktorý mapuje objektový model aplikácie na relačný dátový model databázy.

Cieľ ORM je umožniť programátorovi pracovať s objektmi, zatiaľ čo ORM rieši ukladanie a načítanie týchto objektov z relačnej databázy.

Príklady ORM nástrojov:

- Hibernate,
- JPA,
- Entity Framework,
- Doctrine,
- SQLAlchemy,
- Django ORM.

10.5.1 Čo ORM mapuje

ORM typicky mapuje:

- triedy na tabuľky,
- objekty na riadky,
- atribúty na stĺpce,
- referencie na cudzie kľúče,
- kolekcie na vzťahy 1:N alebo M:N,
- dedičnosť na relačné tabuľky,
- enumy a vlastné typy na databázové typy,
- operácie nad objektmi na SQL príkazy.

Príklad:

```
class Customer {  
    Long id;  
    String email;  
    List<Order> orders;  
}
```

Môže byť mapované na tabuľky:

```
customers(customer_id, email)  
orders(order_id, customer_id, ...)
```

10.5.2 Výhody ORM

1. Práca s objektmi

Vývojár môže pracovať s objektmi a nemusí ručne písať všetky SQL dotazy.

2. Zníženie množstva boilerplate kódu

ORM automatizuje opakujúce sa operácie ako insert, update, delete a select.

3. Mapovanie vzťahov

ORM dokáže mapovať asociácie medzi objektmi na cudzie kľúče alebo spojovacie tabuľky.

4. Prenositelnosť medzi databázami

Niektoré ORM umožňujú jednoduchšie prejsť z jedného DBMS na iný, hoci v praxi to nie je vždy úplne bezproblémové.

5. Unit of Work a zmenový tracking

ORM môže sledovať zmeny objektov a pri commitnutí transakcie ich premietnuť do databázy.

6. Lazy loading

Súvisiace objekty sa nemusia načítať okamžite, ale až vtedy, keď sú skutočne potrebné.

10.5.3 Nevýhody ORM

1. Skrytá zložitosť

ORM môže skrývať SQL dotazy. Programátor nemusí presne vidieť, koľko dotazov sa vykoná a aké sú náročné.

2. N+1 query problém

Pri lazy loadingu môže vzniknúť situácia, že sa najprv načíta zoznam entít a potom sa pre každú entitu samostatne načítajú súvisiace dáta.

Príklad:

```
SELECT * FROM orders;
SELECT * FROM order_items WHERE order_id = 1;
SELECT * FROM order_items WHERE order_id = 2;
SELECT * FROM order_items WHERE order_id = 3;
...
```

3. Výkonové problémy

ORM nemusí generovať optimálne SQL pre zložité dotazy. Pri analytických alebo veľmi špecifických dotazoch môže byť ručne písané SQL vhodnejšie.

4. Zložité mapovanie dedičnosti

Dedičnosť z OO modelu sa do relačných tabuliek mapuje ťažko.

5. Riziko anemického doménového modelu

Pri nesprávnom použití ORM môžu entity slúžiť iba ako dátové kontajnery bez správania, pričom logika je roztrúsená v service triedach.

6. Problémy s transakciami a životným cyklom objektov

Treba riešiť, či je objekt pripojený k session, či sa zmeny sledujú, kedy sa vykoná flush a čo sa stane po zatvorení transakcie.

7. Netesná abstrakcia

ORM sa snaží skryť relačnú databázu za objektový model, ale pri zložitejších prípadoch musí vývojár aj tak rozumieť SQL, indexom, transakciám a relačnému modelu.

10.6 Typické stratégie mapovania dedičnosti v ORM

10.6.1 Single Table Inheritance

Celá dedičná hierarchia sa uloží do jednej tabuľky.

```
products(  
    id,  
    type,  
    name,  
    frame_size,  
    wheel_size,  
    pieces_count,  
    wash_cycles_count  
)
```

Výhody:

- jednoduché dotazy,
- netreba joiny pre podtriedy.

Nevýhody:

- veľa NULL hodnôt,
- horšie constraints,
- tabuľka môže byť neprehľadná.

10.6.2 Class Table Inheritance

Spoločné atribúty sú v tabuľke rodičovskej triedy a špecifické atribúty v tabuľkách podtried.

```
Product(id, name, price)
BicycleProduct(product_id, frame_size, wheel_size)
LegoSetProduct(product_id, pieces_count)
```

Výhody:

- normalizovanejší model,
- menej NULL hodnôt,
- lepšie oddelenie spoločných a špecifických atribútov.

Nevýhody:

- potreba joinov,
- zložitejšie dotazy,
- horší výkon pri veľkých hierarchiách.

10.6.3 Concrete Table Inheritance

Každá konkrétna trieda má vlastnú tabuľku so všetkými potrebnými atribútmi.

```
BicycleProduct(id, name, price, frame_size, wheel_size)
LegoSetProduct(id, name, price, pieces_count)
```

Výhody:

- jednoduché tabuľky pre konkrétne typy,
- dobrý výkon pri dotazoch na konkrétny typ.

Nevýhody:

- duplicita spoločných atribútov,
- zložité dotazy cez všetky typy,
- horšia údržba spoločných pravidiel.

10.7 NoSQL databázy

NoSQL databázy sú databázy s nerelačnou logickou štruktúrou. Presnejšie by sa malo hovoriť o nerelačných databázach, pretože nejde iba o absenciu SQL, ale najmä o to, že logická štruktúra dát je iná než relačné tabuľky. Materiál uvádza typické NoSQL štruktúry: key-value páry, dokumenty, wide-column formát a graf.

10.7.1 Spoločné vlastnosti NoSQL databáz

NoSQL databázy často majú tieto vlastnosti:

- nerelačná logická štruktúra,
- SQL-like alebo vlastný query jazyk,
- návrh pre distribuované prostredie,
- vysoká škálovateľnosť,
- často uvoľnené ACID vlastnosti,
- obmedzená referenčná integrita,
- obmedzenejšie constraints,
- obmedzenejšia zložitosť dotazov,
- schema-less alebo flexibilná schéma,
- časté použitie v cloudovom prostredí.

NoSQL databázy sú často vhodné tam, kde je dôležitá škálovateľnosť, dostupnosť, flexibilita schémy alebo práca s pološtruktúrovanými dátami.

10.8 Typy NoSQL databáz

10.8.1 Key-value databázy

Key-value databáza ukladá dáta ako dvojice:

$\text{key} \rightarrow \text{value}$

Kľúč je unikátny, často textový reťazec. Hodnota môže byť reťazec, zoznam, množina, JSON objekt alebo iný typ.

Príklady operácií:

```
SET product:KA23E '{"name": "BestBike Junior"}'  
GET product:KA23E  
EXISTS product:KA23E  
DEL product:KA23E
```

Použitie:

- rýchle vyhľadávanie podľa kľúča,
- cache,
- session storage,
- jednoduché profily alebo konfigurácie,

- aplikácie vyžadujúce vysokú škálovateľnosť.

Príklady:

- Redis,
- Amazon DynamoDB,
- Couchbase.

Materiál zdôrazňuje, že key-value stores podporujú najmä jednoduché dotazy podľa kľúča.

10.8.2 Document databázy

Document store ukladá dokumenty identifikované unikátnym kľúčom. Dokumenty bývajú vo formáte JSON, BSON, XML alebo YAML.

Príklad dokumentu:

```
{
  "firstName": "Wood",
  "lastName": "Lyons",
  "addresses": [
    {
      "street": "Ralph Avenue",
      "number": 895,
      "city": "Bynum"
    },
    {
      "street": "Surf Avenue",
      "number": 386,
      "city": "Fairlee"
    }
  ]
}
```

Výhody:

- prirodzené ukladanie hierarchických dát,
- menej joinov,
- flexibilná schéma,
- rýchly vývoj webových aplikácií,
- dobrá škálovateľnosť.

Použitie:

- pološtruktúrované dáta,

- katalógy produktov,
- používateľské profily,
- webové aplikácie,
- dáta s vnorenou štruktúrou.

Príklady:

- MongoDB,
- CouchDB,
- Couchbase.

Materiál uvádza aj príklad prevodu JSON dokumentu s adresami na relačné tabuľky **Persons** a **Addresses**. Tento príklad ukazuje, že hierarchická štruktúra sa v relačnom modeli musí rozložiť do viacerých tabuliek.

10.8.3 Wide-column databázy

Wide-column databázy ukladajú dáta podľa riadkových kľúčov a stĺpcových kľúčov. Každý riadok môže mať iné stĺpce. Možno ich chápať ako riedku maticu alebo dvojrozmerné pole key-value párov.

Vlastnosti:

- unikátne row keys,
- rôzne stĺpce pre rôzne riadky,
- dotazy podľa jedného kľúča alebo rozsahu,
- selektívne čítanie stĺpcov,
- vhodné pre veľké objemy dát.

Použitie:

- pološtruktúrované alebo meniace sa dáta,
- riedke dáta,
- time series dáta,
- vysoký throughput čítania a zápisu,
- škálovateľné aplikácie.

Príklady:

- Apache Cassandra,
- Apache HBase,
- Google BigTable.

10.8.4 Grafové databázy

Grafové databázy ukladajú uzly, hrany a ich vlastnosti. Sú vhodné pre dáta, kde sú dôležité vzťahy medzi objektmi.

Použitie:

- sociálne siete,
- odporúčacie systémy,
- znalostné grafy,
- sieťové topológie,
- detekcia podvodov,
- prehľadávanie vzťahov.

Príklad:

- Neo4j.

Materiál uvádza, že grafové databázy efektívne ukladajú uzly, hrany a ich vlastnosti a často sú vysoko škálovateľné.

10.9 Relačné databázy verzus NoSQL

Vlastnosť	Relačné databázy	NoSQL databázy
Logická štruktúra	Tabuľky, riadky, stĺpce	Dokumenty, key-value, wide-column, graf
Schéma	Zvyčajne pevná	Často flexibilná alebo schema-less
Dotazovanie	SQL, silné možnosti joinov	SQL-like alebo vlastné jazyky, často obmedzenejšie dotazy
Integrita	Silná referenčná integrita, constraints	Často obmedzená referenčná integrita a constraints
Transakcie	Typicky ACID	Často uvoľnené ACID vlastnosti
Škálovanie	Tradične vertikálne, dnes aj distribuované riešenia	Často navrhnuté pre horizontálne škálovanie
Použitie	Transakčné systémy, konzistentné dáta, zložité dotazy	Veľké objemy dát, flexibilná schéma, cloud, vysoká dostupnosť

Table 5: Porovnanie relačných a NoSQL databáz

10.10 Výber vhodného DBMS

Pri výbere databázového systému treba zohľadniť viacero aspektov:

- logickú dátovú štruktúru,
- funkčné požiadavky,
- dátové operácie,
- vlastnosti dát, napríklad objem a uniformitu,
- nefunkčné požiadavky,
- náklady,
- projektový rozpočet,
- návrhové obmedzenia,
- požiadavky na škálovateľnosť,
- dostupnosť,
- spoľahlivosť,
- bezpečnosť,
- výkon,
- čas odozvy,
- prevádzkové prostredie.

Materiál osobitne zdôrazňuje, že pri výbere DBMS treba zvažovať logickú dátovú štruktúru, centralizované alebo distribuované uloženie, on-premise alebo cloud databázu a rozdiel medzi transakčným a analytickým spracovaním.

10.11 Centralizované a distribuované databázy

Centralizovaná databáza beží a ukladá dáta na jednom stroji. Distribuovaná databáza beží a ukladá dáta na viacerých počítačoch, prípadne na viacerých fyzických lokalitách.

Dôvody distribúcie:

- odstránenie single point of failure,
- škálovateľnosť,
- dostupnosť,
- spoľahlivosť,

- lepší čas odozvy,
- geografická blízkosť používateľom.

Nevýhody distribúcie:

- vyššia operačná zložitosť,
- sieťová komunikácia,
- náročnejšia konzistencia,
- vyššia learning curve,
- zložitejšie monitorovanie a správa.

Dáta možno distribuovať najmä dvoma spôsobmi:

- **replikácia** – viaceré kópie dát na viacerých uzloch,
- **partitioning alebo sharding** – rozdelenie dát na časti, ktoré sú uložené na rôznych uzloch.

10.12 Kedy použiť relačnú databázu

Relačná databáza je vhodná najmä vtedy, keď:

- dáta majú jasnú štruktúru,
- potrebujeme silnú konzistenciu,
- potrebujeme transakcie,
- potrebujeme referenčnú integritu,
- potrebujeme zložité dotazy a joiny,
- ide o klasický informačný alebo transakčný systém.

Príklady:

- bankový systém,
- účtovníctvo,
- objednávkový systém,
- skladový systém,
- systém evidencie zákazníkov.

10.13 Kedy použiť NoSQL databázu

NoSQL databáza je vhodná najmä vtedy, keď:

- dáta sú pološtruktúrované alebo sa často menia,
- potrebujeme vysokú horizontálnu škálovateľnosť,
- nepotrebujeme zložité joiny,
- chceme ukladať dokumenty alebo hierarchické dáta,
- potrebujeme rýchle lookupy podľa kľúča,
- spracúvame veľké objemy dát,
- aplikácia je cloud-native alebo distribuovaná.

Príklady:

- produktový katalóg s rôznymi atribútmi,
- logovanie udalostí,
- cache,
- session storage,
- používateľské profily,
- sociálne grafy,
- time-series dáta.

10.14 Zhrnutie

Objektovo orientované systémy a relačné databázy sú nekompatibilné v tom, že používajú odlišné modely dát. OO systém pracuje s objektmi, identitou, referenciami, dedičnosťou, správaním a zapuzdrením. Relačná databáza pracuje s tabuľkami, riadkami, stĺpcami, primárnymi a cudzími kľúčmi. Tento rozdiel sa nazýva objektovo-relačný nesúlad.

ORM, teda Object-Relational Mapping, mapuje triedy na tabuľky, objekty na riadky, atribúty na stĺpce a referencie na cudzie kľúče. Uľahčuje prácu s databázou z objektového kódu, ale prináša aj problémy, napríklad skrytú zložitosť, N+1 query problém, zložité mapovanie dedičnosti a výkonové riziká.

NoSQL databázy sú nerelačné databázy. Patria sem key-value stores, document stores, wide-column stores a graph databases. Často sú navrhnuté pre distribuované prostredie, škálovateľnosť a flexibilnú schému, ale zvyčajne majú obmedzenejšiu referenčnú integritu, constraints a zložitosť dotazov.